

qedb: Expressive and Modular Verifiable Databases (without SNARKs)

Vincenzo Botta
vincenzo@provably.ai
Provably Technologies
Roma, Italy

Simone Bottoni
simone@provably.ai
Provably Technologies
Varese, Italy

Matteo Campanelli
binarywhalesinternaryseas@gmail.com
Offchain Labs and University of Tartu

Emanuele Ragnoli
emanuele@provably.ai
Provably Technologies
Warsaw, Poland

Alberto Trombetta
alberto@provably.ai
Provably Technologies
Como, Italy

Abstract

Verifiable Databases (VDBs) allow clients to outsource data storage without trusting the provider: a client holding only a short digest can verify any query response using a compact server-provided proof. Given the ubiquity of both databases and outsourced storage, VDBs address a fundamental need.

Our work advances the state of the art in VDB design. Our main contribution is **qedb**, a simple and performant construction for SQL queries based on bilinear pairings. Like some prior VDB schemes, qedb leverages features specific to the database setting; however, it differs from such approaches in its technical blueprint, the breadth of supported queries, and performance. Notably, it is the first scheme of its kind with proof size independent of database size and without quadratic preprocessing. Compared to VDB solutions based on general-purpose proofs, qedb offers stronger tradeoffs in at least one of the following: provable security, proof size and verification time, or system complexity and maintainability (over an order of magnitude fewer lines of code).

As additional contributions, we provide both an implementation of qedb and new theoretical foundations for VDB design—a novel framework modeling *idealized* protocols for verifiable databases, which future work can use in a plug-and-play manner. Through our modular approach we can get more provably secure instantiations of qedb *for free*, including a post-quantum one from lattices.

1 Introduction

Databases are one of the foundations of modern technological infrastructure. They underpin financial systems, enable researchers to analyze scientific data, support social platforms in storing user activity, and allow governments to manage critical citizen records. Given their central role, guaranteeing their integrity features is fundamental. However, organizations increasingly outsource their databases to third parties, who are trusted both with storing data correctly but also with that of executing queries correctly.

Verifiable Databases (VDBs) [75] offer a cryptographic solution to the challenge above. In a VDB, a client queries an outsourced dataset db and will hold only a compact digest δ_{db} to db . Each time the server receives a query to the dataset—in SQL, say—it will attach a short proof π to its response. If together, δ_{db} , π , and the response R pass a verification check, the client accepts R as valid.

If effective, VDBs can avoid the need for the client to recompute the response or to trust the server. But to be effective *in the*

real-world, a VDB scheme should be *practical* and *secure* along several dimensions. In this work, we are concerned with verifiable databases that offer better practical and security tradeoffs.

In order to be practical, VDBs should be efficient—ideally with fast public verification and short proofs (*succinctness*)—and non-interactive, so that the prover is not required to be online after responding. As for security-related features, solid cryptographic assumptions are essential. Yet, they alone are not a guarantee of real-world security. An additional desideratum of a trustworthy VDB construction, we argue, is that it should be as simple as possible. In software, simpler designs tend to be less vulnerability-prone and easier to audit and maintain [32, 81]. Nonetheless, many efficient VDB schemes—both in prior literature and industrial efforts—come with considerable complexity, which may undermine these very goals.

1.1 Our contributions

Our goal in this work is to *leverage specific aspects* of the database domain to obtain a simple, secure, and performant proof system. We propose a new design for VDBs that: *a)* improves on prior works in terms of both performance tradeoffs and a robust provable security argument; *b)* pushes the limits of what can be proven efficiently in this domain by combining simple building blocks. We also provide the first formal framework for VDB design. In more detail:

- As a main contribution, we describe a new VDB construction called **qedb**, which supports a representative subset of SQL. Our scheme represents queries through a *modular* approach that scales to large databases. The latter approach stems from a novel interplay of two main cryptographic primitives: vector commitments *with inner product features* and *accumulators*. Compared to prior “DB-tailored” approaches¹, qedb: (i) is **more expressive**; (ii) achieves **proof size completely independent** of database size; (iii) and **avoids quadratic behavior** for both database processing and storage (a dependency that in prior work leads to 100–225× overhead on common table sizes²). All these properties are obtained thanks to our techniques. When compared to VDBs built from general-purpose proof systems—a

¹As we discuss in the related work, designs for VDBs fall roughly into two categories: (i) those explicitly leveraging the structure of this domain by employing specialized (“tailored”) cryptographic primitives; (ii) those employing general-purpose proof systems. This is not a sharp taxonomy; some approaches can be in between categories.

²This is a direct implication of the quadratic behavior in table with just 10–15 columns.

second family of solutions characterized by being highly expressive but also generally more complex—qedb offers at least one of the following: a simpler architecture using only basic building blocks—that does not require writing circuits—with over one order of magnitude fewer lines of code compared to some schemes, competitive or superior performance, or stronger security guarantees through fewer, better-understood cryptographic assumptions.

- Along with our new construction, we develop **the first abstract framework for VDB constructions**. We thus bring VDB design closer to established SNARK practices that separate “idealized” information-theoretic blueprints from a “cryptographic compilation step”. Our framework is novel and diverges from those commonly used in general SNARKs—based on polynomial evaluation [31] or random access to strings [5]—by being specialized to settings with a database flavor by employing minimalist-but-structured oracles to vectors and sets³. Through our modular approach, **we can obtain additional provably secure instantiations of qedb (for free)**, including a post-quantum one from lattices.
- **We implement the bulk of qedb as a practical software system⁴**. Besides the cryptographic components, our implementation includes a middleware layer that can be easily integrated with existing databases to make them verifiable. We also experimentally confirm that the qedb’s advantages on paper translate to concrete performance. On *representative queries*, our experiments show: proof sizes below 1KB and verification of 6–20ms (with slightly higher numbers on specific queries); a prover running on a common laptop in a handful of seconds on DBs with tens of thousands of rows. Moreover, we show that qedb can scale to tables with millions of rows (on a common laptop), whereas prior state-of-the-art solutions in the same category (IntegriDB [96]) fail at $\approx 100K$ rows due to memory constraints.

Our final construction can be instantiated from already deployed powers-of- τ setups used for the KZG polynomial commitment [57] and other popular SNARKs. The supported queries of our construction include SELECT and aggregate queries (SUM, MAX, COUNT, ...) with different types of predicates (multi-dimensional ranges, membership in lists, ...), JOINs, GROUP BY, and others. In terms of expressivity, our scheme improves on state-of-the-art schemes in the same family by, for example, supporting comparison and combination of columns (see, e.g., query in Fig. 3 and an expanded list in Fig. 14 in the appendix), a common query pattern in real-world SQL.

1.2 Motivating applications

In addition to their natural applicability in general database settings, we highlight some concrete applications of VDBs.

Financial data reporting. Consider a financial firm that curates a database market transactions, upon which a third-party reporting system runs analytics queries on behalf of external clients, such as

³In the literature on SNARKs, “oracle to X ” usually refers to some idealized way of formalizing access to X . The reader unfamiliar with this literature can simply think of oracles as a loose analogue of “pointers to X ” (in the programming languages sense).

⁴We are planning to make it available as open source soon.

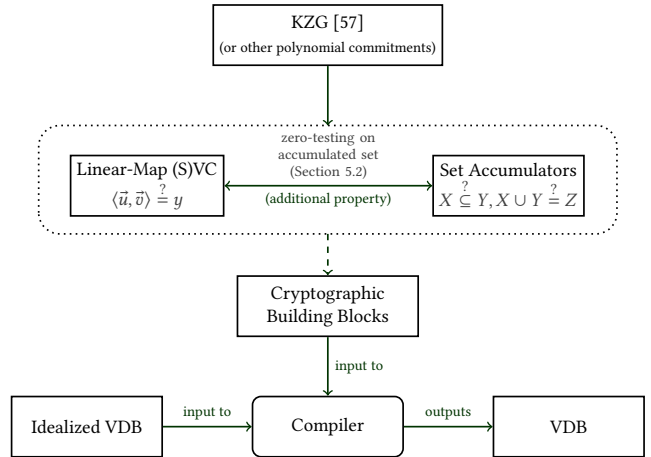


Figure 1: Instantiating VDBs in our abstract framework.

regulatory entities⁵. We provide a simple representative example of queries for this scenario supported by qedb in Fig. 2. The database assumed in the figure contains at least three tables Account, Asset, Transaction that store data about clients’ accounts, traded assets, and transactions between accounts involving such assets. The setting of financial data reporting showcases some of the features of qedb: in this scenario, it is crucial that such aggregate data reporting is trustworthy and compliant with regulation [41][39]. Simpler constructions such as qedb are easier and cheaper to audit, and thus easier to demonstrate as compliant.

Data market aggregator. An untrusted data aggregator analyzes data from multiple providers. The analyses may use several metrics and are accessible to data consumers, certifying the validity of Data Service Level Agreements, e.g. *all the records in the reporting data sets must have been updated within the prior 24 hours* [33, 73].

Blockchain oracles and blockchain analytics systems. The role of a blockchain oracle [50] is to feed data coming from off-chain sources to smart contracts operating on-chain. Assuming that data sources are trusted, it is crucial that an oracle proves to the blockchain that data has not been tampered with [74]. As an additional viewpoint on the last setting, consider a datastore containing a curated list of transactions executed on a blockchain. Clients are interested in executing analytics-based queries on such data. Here again, the client can be convinced of the validity of the responses without having to process the data themselves [42].

2 Technical Overview

At the highest level, our work leverages appropriately designed *authenticated data structures* (or ADS), simple cryptographic primitives with specialized proving capabilities. Popular schemes in this family include polynomial commitments [57], set accumulators [6], and vector commitments [24]. This design choice is motivated by avoiding general-purpose proof systems (such as SNARKs or “zk”VMs) and obtaining a simpler, more auditable, and maintainable solution. The state-of-the-art for VDBs of this kind is IntegriDB [96].

⁵As a concrete instance of this scenario: in the US, firms such as JPMorganChase and HSBC report annually to the US Securities and Exchange Commission on their financial and compliance status, regulatory capital, and risk metrics [52, 56].

Q_{Tot} || `SELECT SUM(price) FROM Transaction`
|| `WHERE account_id = '5938' AND trade_date = '2025-01-01'`
 $\vdash$ Computes total price of transactions executed by an account on a given date

 Q_{CntTx} || `SELECT COUNT(*) FROM Transaction`
|| `WHERE trade_date BETWEEN '2025-01-01' AND '2025-03-31'`
 $\vdash$ Computes the number of transactions executed within the first quarter

 Q_{Match} || `SELECT tx_id, price, expected_price, price=expected_price`
|| `FROM Transaction WHERE trade_date = '2025-04-05'`
 $\vdash$ Retrieves the transactions whose executed price matches their expected price

Figure 2: Example queries for financial data reporting.

IntegriDB supports multiple query types and offers relatively succinct proof and verification (with $|\pi| \approx$ tens of KBs and verification time $\approx$ tens or hundreds of milliseconds). We now highlight some of the key ideas in IntegriDB and their limitations. We assume basic familiarity with relational databases and SQL; we refer the reader to, e.g., [4] for a quick introduction.

Background: the state of the art in verifiable databases from ADS and its limitations. Simply put, at the heart of IntegriDB is the observation that one can express a substantial subset of SQL by combining two primitives: *i*) set accumulators from bilinear pairings (accumulators are an ADS providing short commitments to sets and related proving capabilities) and that can succinctly certify operations on sets such as union, intersection, etc.; *ii*) an authenticated interval tree (which can efficiently certify that a node x in the tree is such that the key associated to x is within an interval $[a, b]$). In a nutshell and simplifying, the key idea is to employ the accumulators to encode sets of *values* in the columns of the DB and to use accumulators as nodes in the authenticated interval tree.

This specific approach brings about several limitations:

- *Additional machinery and loss of succinctness for duplicates:* values in a table might naturally appear on multiple rows; however, accumulators cannot capture this fact, requiring ad-hoc handling and additional bandwidth on certain queries.
- *Proof size dependent on $|\text{db}|$:* because proving an interval query requires paths in the tree (which grows with the size of the database) larger databases yield larger proofs.
- *Quadratic storage and preprocessing and limitations in expressivity:* this approach requires being able to preprocess all possible ways we might match columns; it also cannot be used for queries that *combine* columns respecting the order of rows (lost by committing to *sets*⁶).

A final limitation is a lack of generality. The description of IntegriDB’s blueprint is in terms of *specific* instantiations (bilinear pairing accumulators and authenticated interval trees of a specific type). Thus, it is not clear how its security and correctness changes when one replaces one building blocks with another (and whether one can). Our methodology for qedb addresses all these limitations.

Towards new building blocks for VDBs. The limitations we just saw for IntegriDB stem mainly from the heavy reliance on sets as the main underlying abstraction. When a column is encoded as a set, it loses several of its natural features, such as being ordered

and admitting repeated elements. Thus, one technical question that drives our work is: *What are other suitable abstractions (and related cryptographic primitives) for proving DB queries?*

When searching for abstractions that can capture the encoding of *columns*, an intuitive candidate is clearly provided by *vectors*—ordered sequences of values—and their cryptographic counterpart, *vector commitments* [25]⁷. Through this type of cryptographic primitives a client, holding a commitment $\text{cm}_{\vec{v}}$ to a vector $\vec{v}$, can verify that a tuple $\vec{u}$ corresponds specifically to the positions $(v_i)_{i \in I}$ for any subset of indices I (this property is known as *subvector opening*; it is useful especially because $\vec{u}$ can be much smaller than $\vec{v}$). Vector commitments have also been formalized and constructed extending them with the ability of proving inner products [22] (i.e., to prove that $\langle \vec{v}, \vec{w} \rangle$ yields a target value y). The resulting primitive is called a Linear-map Vector Commitment (LVC), which we will employ.

At this point, we can already give an intuition on a potential approach from a vector commitment with both subvector opening and linear-map features (we simply refer to this combined primitive as “VC”). During preprocessing time—the one-time offline stage where a database is “cryptographically indexed” for later verifiable queries—we can commit to each column col. The verifier can later access this material during proof verification⁸. An example of how we can use this to prove/verify a simple SUM query is on the right side of Fig. 3. There, we employ an additional property common to VC constructions, *homomorphism*, through which we can combine commitments via linear functions (*intuition*: the sum of two commitments is a commitment to the sum of the respective openings). The query we target is at the top of Fig. 3; we stress this is a query not supported by IntegriDB, which our approach unlocks.

Challenges: expressivity, succinctness, and the interplay between vectors and sets. So far, we only exemplified how vectors can work in our favor as an abstraction on a *rudimentary* query. Imagine extending the query in Fig. 3 with “ [...] WHERE Price $\geq$ 100”. Now the prover needs to show that S corresponds, not to the sum of all vector values, but of those in a particular set of rows X (the ones where price values are at most 100). If we want to obtain a scheme with proofs of size independent of the database size, we cannot send the values in the rows in X or otherwise a super-constant number of bits (recall that what the client requested is a single number S). A promising avenue for solving this problem comes from reintroducing sets as an abstraction (and set accumulators with them). In contrast to IntegriDB, however, when filtering rows, we will use accumulators as digests to sets of rows, not values. This way, as part of the proof, the server can also send an accumulator acc_X committing to the set of rows X . But we are not quite done yet—we need to figure how acc_X can be used, which is not obvious. Through it, the client, should somehow be able to both check: *(i)* that the value S is computed as the sum of the indices opened by acc_X , and *(ii)* that the set behind acc_X is effectively the right one

⁷Some remarks on this observation and the technical novelty of our work are in order. While it may seem obvious *in hindsight*, no academic work has exploited it so far despite the explosion of research in vector commitments. Also, it is not sufficient per se—as we discuss in this overview, we need to solve additional challenges.

⁸In our scheme the final commitment to the whole database held by the client will *in turn* consist of a vector commitment to all the commitments to the columns and other material produced at pre-processing. For simplicity we ignore this detail here.

⁶An example of such queries is at the top of Fig. 3; see also Appendix F.

(i.e., it contains *all and only* the indices i such that $p_i \geq 100$). In fact, we must be able to answer these two questions:

- a) can we (succinctly) prove that the opening of an LVC refers to a specific set of *accumulated* positions? (necessary for (i)),
- b) how can we prove complex properties (such as range checks and “all and only”-type statements) *exclusively* from functionalities related to VC and accumulators? (necessary for (ii)).

We answer *a*) in the positive in Section 5.2 (and Appendix D), where we formalize and instantiate a new property that links VC and set accumulators. This property provides a versatile *interface* between vectors and sets and (through additional tricks) will indirectly help us tackle (i) and similar settings—we refer to it as *zero-testing on accumulated sets* as it permits to succinctly verify “ $\forall i \in X v_i = 0$ ” given $\text{cm}_{\vec{v}}$ and acc_X . We now discuss how to address challenge *b*).

An expressive idealized framework for VDBs. We are able to show how the cryptographic functionalities we have so far—subvector and linear-map opening, set related operations and our new zero-testing property—can *alone* capture a non-trivial subset of SQL (see examples in Fig. 2 and a full list is in Appendix F). However, instead of proceeding by *directly* showing the expressivity of our *cryptographic* components, we take an alternative route: we design an *idealized* framework that captures the role of vectors and sets as *abstractions* by modeling them as information-theoretic oracles (see example in the left side of Fig. 3 and description in Fig. 8). We then provide a collection of “utility” operations that can be derived unconditionally (information-theoretically) in our framework (Fig. 9 and Section 6.1). This collection includes complex items tantamount to range proofs and generalized preimage checks (showing an accumulator “contains” all and only the rows whose value is a target value α). Our methodology is motivated by three goals: maintaining a clear separation of concerns between the blueprint behind a VDB (the *essence* of its approach) from its cryptographic implementation; providing a theoretical foundation for VDB design that future work can build upon (see also discussion of our compiler below); being able to formalize our protocol through simple building blocks that are trivially correct (as they do not rely on algebra and closely follow the SQL operations we target).

The last piece of our general framework is a compiler that transforms a *succinct* idealized VDB into a *succinct* cryptographic VDB. Our main compilation theorem—see Section 5.2—shows that VC and accumulator with appropriate properties can compile an idealized VDB into a cryptographic protocol (see again Fig. 3 and diagram in Fig. 1); we also show how homomorphic polynomial commitments suffice for this compilation. Our final construction, qedb^9 —of which we provide a partial overview in Section 6—is obtained from this framework and the way we apply it specifically to the SQL setting.

3 Related Work

Works on VDB can be (roughly) categorized into two broad categories, depending on the underlying approach: from authenticated data structures or from general cryptographic proof systems¹⁰

Prior schemes from authenticated data structures (ADS). To the best of our knowledge, after almost a decade, IntegriDB [96] still essentially represents the state of the art for efficient and expressive approaches based on ADS with succinct proofs. The reader can find a comparison of qedb and IntegriDB in Section 7.2. Other relevant works can be categorized as tree-based (e.g., [62]) or signature-based (e.g., [71]). None of these works supports a generally efficient approach for JOIN queries. In [69], the authors present a general method for compiling a description of a data structure into its authenticated counterpart from collision-resistant hashing. As such, this approach cannot take advantage of the optimizations that are possible using algebraic commitments. Our work can be seen as adopting a similar abstract angle as that in [69] but with a more specialized, algebraic-flavored approach (instead of λ -calculus) and resulting in more succinct schemes. Additional references are in the survey [83]. We finally cite representative works on functional commitments, a primitive close in spirit to ADS [8, 16, 19, 25, 26, 38, 57, 60, 65, 66, 90, 91].

Prior schemes from general cryptographic proof systems. vSQL [95] is a system that provides verifiability for a large subset of SQL queries. At its essence, it can be considered the earliest example of VDB from general-purpose proof systems (it is based on (an optimization of) the GKR protocol [46]). For a comparison to our work, we refer the reader to Section 7.3. We mention two recent works that, in addition to query verifiability, support zero-knowledge. ZKSQL [63] is VOLE-based [40] (thus inherently interactive and supporting non-public verification only). It supports a large subset of SQL queries, testing over the TPC-H benchmark. Its performances incur in large overheads as the dataset grows over half a million rows. This scheme is not succinct. PoneglyphDB [48] is based on the Halo2 proof system [94] and supports arbitrary SQL queries. It shows better performance than ZKSQL with an asymptotically logarithmic verifier and proof size (improving on the polylogarithmic dependency in vSQL). From preliminary estimates, PoneglyphDB shows typical proving times and proof sizes comparable to ours; on the other hand, it shows verification times at least an order of magnitude worse due to their use of inner product arguments [9] even on relatively small table sizes. While overall a performant proof system supporting zero-knowledge, PoneglyphDB requires writing SQL logic as constraints, and its inner workings are quite complex, involving for example, polynomial constraints verification within the circuit (see discussion in Section 5.5. in [48]). Being based on Halo2, PoneglyphDB relies on recursion. The recent work in [78] provides a VDB leveraging several tech stacks including Jolt, Halo2 and RiscZero. Other approaches, tracing back to [68], provide zero-knowledge to very simple queries over key-value datasets [10, 27, 64] (these last works can be seen as ADS-based ones, but we cite them here in the larger context of zero-knowledge database-like schemes). Verifiable databases have also received recent attention from commercial efforts [3, 59, 85]; our comparisons in Sections 7.3 and 7.4 apply to all of these works.

4 Background

Additional details for this section are in Appendix A.

Asymptotic notation. We write $\tilde{O}(T)$ to denote $O(T \text{polylog}(T))$. $O_\lambda(T)$ (resp. $\tilde{O}(T)$) denotes $O(T \text{poly}(\lambda))$ (resp. $\tilde{O}(T \text{poly}(\lambda))$). We

⁹ qedb is a recursive acronym standing for “ qedb error-checks databases”. It is also a shameless pun on it being a *proof* system for DBs.

¹⁰Besides this section, we compare qedb against other schemes in Section 7.

SELECT SUM(2*Price + ExtraFee) as S FROM PriceInfo

<i>Idealized operations</i>	<i>Cryptographic translation</i>
Pre-processing: Store slices $\tilde{p}$ (Price), $\tilde{x}$ (ExtraFee).	Pre-processing: $\text{cm}_p \leftarrow \text{CommitVec}(\text{prk}, \tilde{p}); \text{cm}_x \leftarrow \text{CommitVec}(\text{prk}, \tilde{x})$.
At query time:	At query time:
- Prover computes and sends answer $S \leftarrow \sum_i h_i$ where $\tilde{h} := 2\tilde{p} + \tilde{x}$. - Client computes $\tilde{h} = 2\tilde{p} + \tilde{x}$ (via homomorphism of slices). - Client accepts iff $\langle \tilde{h}, \tilde{1} \rangle \stackrel{?}{=} S$ (via inner product test).	- Prover sends answer S and proof $\pi_S \leftarrow \text{OpenLin}(\text{prk}, \tilde{h}, \tilde{1}, S)$. - Client computes $\text{cm}_h \leftarrow 2\text{cm}_p + \text{cm}_x$ (via commitment homomorphism). - Client accept iff $\text{VfLin}(\text{vrk}, \text{cm}_h, \text{cm}_1, S, \pi_S) = 1$.

Figure 3: From query to protocol—a glimpse of how our approach works on a rudimentary query. See Section 6.2 and Appendix E. $\tilde{1}$ is the vector $(1, \dots, 1)$; its commitment is assumed to be available from preprocessing. Oracles to vectors (slices) are in cyan. NB: this is *limited* example and does not showcase the more challenging interplay between vectors and sets (see Section 2).

write $\text{negl}(\cdot)$ to denote a function that is asymptotically negligible. A quantity is *overwhelming* if $1 - \text{negl}(\lambda)$ is a lower bound for it.

Algebraic preliminaries. We denote by $\mathbb{H} = \{x_1, \dots, x_n\}$ a multiplicative subgroup in a finite field $\mathbb{F}$. We will use the interpolation of a vector $\vec{v} \in \mathbb{F}^{\bar{n}}$ as a polynomial $p(X) = \sum_i v_i \lambda_i(x)$, where $(\lambda_i)_{i \in [\bar{n}]}$ are the Lagrange polynomials associated to $\mathbb{H}$. We assume familiarity with elliptic curves endowed with a bilinear pairing $e(\cdot, \cdot)$. Multiplication by x in $\mathbb{G}_1$ (resp. $\mathbb{G}_2$) is denoted by $[x]_1$ (resp. $[x]_2$).

Computational assumptions. The q -DLOG assumption states that even when given the public parameters and all group elements corresponding to the first q powers of a secret τ , no efficient algorithm can recover τ . The q -BSDH assumption states that, given the same kind of information about τ , no efficient algorithm can produce any valid pair $(c, \frac{1}{\tau-c} e([1]_1, [1]_2))$, for any choice of c .

The Algebraic Group Model (AGM). The algebraic group model [43] is an idealized model of computation for security analysis sitting between the standard model and generic groups [82]. It can be seen as a generalized knowledge assumption [36] stating that an algebraic adversary $\mathcal{A}$ must know a linear representation of an output group element with respect to input group elements. Notably, a group element’s creation can depend on the group description and the bit representation of known group elements.

Cryptographic building blocks. We will use Authenticated Data Structures (ADS) in our design. The syntax of these primitives directly reflects their function: they can commit to an object and then prove properties of that object. In addition to the proof, the verifier holds only a short commitment to the object, a (short) description of the property and the proof. We are interested in ADS that are *succinct* and that in particular feature $O_\lambda(1)$ -sized commitments and proofs. We will employ specifically three classes of ADS: polynomial commitments (which can prove evaluation of polynomials); vector commitments with both subvector-opening (proving succinctly an arbitrary subset of values is in specific positions) and linear-map capabilities (proving the result of an inner product between committed vectors); set accumulators (which can commit to, or *accumulate*, sets and that can prove set relations among them). In our construction, all the ADS will use the same setup and thus we will describe a *unique* setup and assume this is the one used by all primitives in our descriptions. The syntax of these primitives should be intuitive from their function and we do not describe them

further here, but refer the reader to Appendix A; there, we also describe their required properties, which include correctness, binding, homomorphism and security (extractability).

Below and in Fig. 4 we briefly review some of the instantiations we will employ; they are all based on bilinear pairings. We denote by $\bar{n}$ a size parameter. This will reflect an upper bound on the size of the committed object, respectively the degree of polynomials, size of vectors, and cardinality of sets.

Unique Setup. The setup follows the standard KZG procedure. This setup is used by all primitives used in qedb (see also Fig. 4).

$\text{Setup}(1^\lambda, \bar{n}) \rightarrow (\text{prk}, \text{vrk})$: Generate the bilinear group description $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda)$; Sample $\tau \leftarrow_{\$} \mathbb{F}$; output the pair: $\text{prk} = (\{[\tau^i]_{1,2}, [\lambda_i(\tau)]_1\}_{i=1}^{\bar{n}})$, $\text{vrk} = ([1]_{1,2}, \{[\tau^i]_2, [\lambda_i(\tau)]_2\}_{i=1}^{\bar{n}})$. *KZG Polynomial Commitment Scheme (PCS)* [57]. Theorem 4.1 follows from [31, Appendix B.3] and [12, Section 7].

THEOREM 4.1. *The KZG scheme is a secure PCS in the AGM.*

Linear-map Vector Commitment (LVC). In Fig. 4 we provide an extension of the LVC from [22]. Further discussion and the proof of the next theorem are in Appendix A.4.

THEOREM 4.2. *The scheme in Fig. 4 is a correct, binding, extractable homomorphic LVC in the AGM under the q -BSDH assumption.*

Subvector-Opening Vector Commitment. We extend the LVC with $(\text{OpenSub}, \text{VerifySub})$ to support subvector openings. This construction follows [22, Sec. 6.3, App. C] and is originally from [88].

THEOREM 4.3 (IMPLICIT IN [22, 88]). *The subvector VC in Fig. 4 is correct, binding, homomorphic and extractable in the AGM.*

Set Accumulators (SA). We describe our instantiations of set accumulators in terms of polynomial checks (and, indirectly, polynomial commitments) in Fig. 5.

We employ variants of the accumulators from [76] which, intuitively, encode a set through its characteristic polynomial. This implies that, like in the case of vector commitments (see Fig. 4), a commitment to a set corresponds to a KZG commitment on a specific polynomial; specifically, one accumulates a set S through $P_S(x) = \prod_{s \in S} (x - s)$. This SA supports set unions, intersections, subsets and complements.

KZG Polynomial Commitment	Linear-Map VC	Subvector-Opening VC
Commit ($\text{prk}, \phi(x) \rightarrow (\text{cm}, \text{aux})$: Output $(\text{cm} = \sum_{i=1}^n \phi_i[\tau^i]_1, \text{aux} = \phi(x))$.	OpenLin ($\text{prk}, \vec{u}, \vec{v}, y \rightarrow \pi$): - Find R, H such that $\deg(R) < n-1$ and $\left(\sum_{i=1}^n u_i \lambda_i(x)\right) \left(\sum_{i=1}^n v_i \lambda_i(x)\right) - \tilde{n}^{-1} y = xR(x) + t(x)H(x)$. - Define $\hat{R}(x) = x^2 R(x)$ and $\widetilde{\text{cm}}_v = \sum_{i=1}^n v_i [\lambda_i(\tau)]_2$. - Output $\pi = ([R(\tau)]_1, [H(\tau)]_1, [\hat{R}(\tau)]_1, \widetilde{\text{cm}}_v)$.	OpenSub ($\text{prk}, \text{aux}, \vec{u}, I \rightarrow \pi$: - Let $I = \{j_1, \dots, j_\ell\}$ and $\vec{v} = (v_1, \dots, v_\ell)$ be the subvector of $\vec{u}$ indexed by I, i.e., $v_i = u_{j_i}$ for $i = 1, \dots, \ell$. - Let $u(x)$ be the interpolation of $\vec{u}$ in the domain $\mathbb{E}$, $v(x)$ the interpolation of $\vec{v}$ in the domain I. - Define $t_I(x) = \prod_{i \in I} (x - h_i)$ and find $H(x)$ such that $u(x) - v(x) = t_I(x)H(x)$. - Output $\pi = [H(\tau)]_1$.
OpenPos ($\text{prk}, \phi(x), i \rightarrow (i, \phi(i), \pi_i)$: - Compute $Q(x) = \frac{\phi(x) - \phi(i)}{x - i}$; - Compute $\pi_i = \sum_{j=1}^n Q_i[\tau^j]_1$; - Output $(i, \phi(i), \pi_i)$.	VfLin ($\text{vrk}, \text{cm}_u, \text{cm}_v, y, \pi \rightarrow 0/1$: - Parse $\pi = ([R]_1, [H]_1, [\hat{R}]_1, \widetilde{\text{cm}}_v)$; - Let $Y := e(\tilde{n}^{-1} y[1]_1, [1]_2)$ and accept only if all checks pass: $e(\text{cm}_u, \widetilde{\text{cm}}_v) - Y = e([R]_1, [\tau^2]_2) + e([H]_1, [t(\tau)]_2)$ $e([R]_1, [\tau^2]_2) = e([\hat{R}]_1, [1]_2)$ $e(\text{cm}_v, [1]_2) = e([1]_1, \widetilde{\text{cm}}_v)$.	VfSub ($\text{vrk}, \text{cm}_u, \vec{v}, I, \pi \rightarrow \{0, 1\}$: - Compute $[t_I]_2 = [t_I(\tau)]_2$ and $v(x)$ as above. - Accept only if: $e(\text{cm}_u - [v(\tau)]_1, [1]_2) = e([H]_1, [t_I]_2)$.
VfPos ($\text{vrk}, \text{cm}, i, \phi(i), \pi_i \rightarrow 0/1$: Accept only if this check holds: $e(\pi_i, [\tau]_2 - i[1]_2) = e([1]_1, \text{cm} - \phi(i)[1]_2)$		
VC Commitment Algorithm CommitVec ($\text{prk}, \vec{a} \rightarrow (\text{cm}, \text{aux})$: Output $\text{cm} = \sum_{i=1}^n a_i [\lambda_i(\tau)]_1$ and $\text{aux} = \vec{a}$.		

Figure 4: Instantiations we adopt for polynomial commitments and vector commitment with linear and subvector openings.

For the opening and verification of $\subseteq, \cup, \cap$, and $\setminus$, we rely on the following subprotocols: Quotient: To prove that a polynomial $P(x)$ vanishes on a set of points $T = \{t_1, \dots, t_\ell\}$, it suffices to show that the vanishing polynomial $Z_T(x) = \prod_{t \in T} (x - t)$ divides $P(x)$, i.e., $P(x) = q(x) \cdot Z_T(x)$ for some $q(x)$. Given a set $S = \{s_1, \dots, s_\ell\}$ with $P_S(x) = \prod_{s \in S} (x - s)$, the Prover computes $q(x) = P_S(x)/Z_T(x)$ and commitments $[P_S(\tau)]_1, [Z_T(\tau)]_1, [q(\tau)]_1$. Using Fiat-Shamir, the Prover derives r, evaluates $y_p = P_S(r)$, $y_q = q(r)$, $y_z = Z_T(r)$, and provides openings π_p, π_q, π_z. The Verifier computes r, checks π_p, π_q, π_z and that $y_p = y_q \cdot y_z$. For the Schwartz-Zippel lemma this procedure is sound. Subset: Observe that $T \subseteq S$ is equivalent to $P_T(x) \mid P_S(x)$. The Prover thus computes $q(x) = P_S(x)/P_T(x)$ and sends $([P_S(\tau)]_1, [P_T(\tau)]_1, [q(\tau)]_1, y_s, y_t, y_q, \pi_s, \pi_t, \pi_q)$, verified as in the Quotient protocol. Set difference: Given S and T, define $P_{S \setminus T}(x) = P_S(x)/P_T(x)$, which encodes $S \setminus T$. The proof follows the same structure as for Subset. Intersection: For $I = S \cap T$, correctness requires (i) $I \subseteq S$ and $I \subseteq T$, and (ii) $(S \setminus I) \cap (T \setminus I) = \emptyset$. Condition (i) is proven via Subset. For (ii), following [76, Lem. 6], there exist q_1, q_2 such that $q_1(x)P_{S \setminus I}(x) + q_2(x)P_{T \setminus I}(x) = 1$. The Prover sends $([q_1(\tau)]_1, [q_2(\tau)]_1)$ and openings at r for $P_{S \setminus I}, P_{T \setminus I}, q_1, q_2$. The Verifier checks subset conditions and verifies $q_1(r)P_{S \setminus I}(r) + q_2(r)P_{T \setminus I}(r) = 1$. Complement (only for known size $\tilde{n}$): For universe $U = [\tilde{n}]$, the complement of S is $U \setminus S$, verifiable as in the Set Difference protocol. Union (on sets of maximum size $\tilde{n}$): Intuition: Using De Morgan's law, $S \cup T = \neg(\neg S \cap \neg T)$, where $\neg$ is implemented via the Complement protocol. ^aIn our scheme, this is used only for sets of rows which satisfy this requirement (their number is known).	
---	--

Figure 5: An instantiation of set accumulators from polynomial commitments.

THEOREM 4.4. *There exists an accumulator scheme with $O_\lambda(1)$ commitments and proofs that is correct, binding and extractable in the AGM under the q -BDH assumption and supports subset, union, intersection and complement checks.*

Cryptographic Verifiable Database. A cryptographic Verifiable Database (VDB) consists of four probabilistic polynomial-time (PPT) algorithms: Setup, which generates public parameters; a pre-processing algorithm PreProc that computes a digest of the database; AnsQry, which outputs a query answer and a proof; an algorithm VerQry for verification. The Setup algorithm is run by a trusted third party and the PreProc algorithms, usually run by the *Data Owner*, the party that outsources the database. Informally, a VDB is (i) *complete* if, using a correct setup, AnsQry outputs the correct answer and a proof accepted by VerQry; (ii) (computationally) *sound* if, for all adversarially generated databases, only correct answers admit valid proofs. The formal definitions are a minor variation of those from [75] and are in Fig. 6. We assume an algorithm ValidDB that, given a database db and a size parameter n checks whether db is a database of valid size. We now define succinctness: proofs in a *succinct* VDB impose no substantial overhead on the verifier. Below, $|q|$ is defined as the number of columns q references including those referenced implicitly (e.g., via $*$ in “SELECT * ...”).

DEFINITION 1 (SUCCINCTNESS OF A VDB). *A VDB is succinct if, for any database and query q , $|\pi|$ and $\text{Time}(\text{VerQry})$ are bounded by $\tilde{O}_\lambda(|a| + |q|)$ (where a is a correct response and π is the honest proof).*

5 A General Methodology for VDBs from New Idealized Protocols and Building Blocks

Here we give an overview of idealized VDBs and then describe how to compile them into cryptographic ones *preserving succinctness*.

5.1 Idealized Protocols for VDBs

In this section, we present an information-theoretic analog of the cryptographic VDB primitive described in Fig. 6. Our goal is to somewhat *idealize* this notion in order to make it simpler to construct and to reason about. The interaction flow of an idealized VDB closely matches that of its cryptographic counterpart. It consists of: a one-time stage where the database is preprocessed¹¹; a non-interactive proving algorithm providing a series of “messages” certifying the response to a query; a decision algorithm through which the client can accept or reject the claimed response (see Fig. 7). Idealized and cryptographic VDBs, however, differ in:

¹¹We also refer to this stage as *indexing* in the idealized VDB setting, consistently with the nomenclature common in idealized models for SNARKs [31].

$\text{Setup}(1^\lambda, n) \rightarrow (\text{prk}, \text{vrk})$: On input λ and size parameter n outputs a pair of keys (prk, vrk) (proof and verification keys). $\text{PreProc}(\text{prk}, \text{db}) \rightarrow (c, \text{state})$: On input prk, and a valid database db deterministically produces a (public) commitment c and some internal state information state. $\text{AnsQry}(\text{prk}, q, \text{state}) \rightarrow (a, \pi)$: On input a query $q \in Q$, prk, and state deterministically outputs an answer/proof pair (a, π), where $a = Q(q, \text{db})$. $\text{VerQry}(\text{vrk}, q, a, c, \pi)$: On input V, the commitment c, a query q and an answer/proof pair (a, π) deterministically outputs “accept” or “reject”.	Completeness: For all $\lambda, n \in \mathbb{N}, q \in Q$, db s.t. $\text{ValidDB}_n(\text{db}) = 1$, the following is overwhelming: $\Pr \left[\begin{array}{l} \text{VerQry}(V, q, a, c, \pi) = 1 \\ \wedge a \in Q(q, \text{db}) \end{array} : \begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda); (c, \text{state}) \leftarrow \text{PreProc}(\text{prk}, \text{db}) \\ (a, \pi) \leftarrow \text{AnsQry}(q, \text{state}) \end{array} \right]$ (Computational) Soundness: For all $\lambda, n \in \mathbb{N}$, PPT $\mathcal{A}$, the following is overwhelming: $\Pr \left[\begin{array}{l} \text{ValidDB}_n(\text{db}) = 1 \wedge \\ \forall i \in [t] (b_i = 1 \Rightarrow \\ a_i \in Q(q_i, \text{db})) \end{array} : \begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n); (\text{db}, \{(q_i, a_i, \pi_i)\}_{i \in [t]}) \leftarrow \mathcal{A}(\text{prk}) \\ (c, \cdot) \leftarrow \text{PreProc}(\text{prk}, \text{db}) \\ b_i \leftarrow \text{VerQry}(\text{vrk}, q_i, a_i, c, \pi_i), i \in [t] \end{array} \right]$
--	---

Figure 6: Syntax and security properties of cryptographic (non-interactive) VDBs. $Q(\cdot, \cdot)$ is the set of valid responses to a query.

- *The capabilities and syntax of the prover.* An adversary in the idealized model is computationally unbounded. Additionally, both honest and adversarial provers are modeled as being able to send to the verifier a string as well as *oracles*, called *handles*, to vectors and sets (elements in cyan).
- *The capabilities and syntax of the verifier.* When deciding whether to accept or reject, the verifier has special access to tests and operations (explained below) to both the handles received from the prover and the ones resulting from preprocessing (which is also restricted accordingly as to output handles only).

When designing the role of handles in this model, our motivation was two-fold: capturing what seem to be essential (and cryptography-friendly) *abstractions* for VDB blueprints, and capturing *succinctness*. We now expand on both.

From databases to handle abstractions. As motivated in Section 2, if used appropriately, sets and vectors are particularly apt to encode the data stored inside a database (e.g., expressing columns as vectors) and for representing queries over the database (e.g., by representing filtered sets of rows as sets). Here we model verifier’s access to sets and vectors as “immaterial pointers” (handles) which allows the verifier to access them through a specific interface but without actually storing these objects. Our model thus provides two types of handles (see top of Fig. 8):

- *Set handles:* given a set X of arbitrary values, we denote the set handle of X is denoted as X. Intuitively, sets in a handle may capture *actual values* in a DB, or *indices to rows* in columns¹².
- *Slice handles:* Given a vector $\vec{v}$ of values, we denote the slice handle of $\vec{v}$ as v. For indexing, slice handles usually contain columns in a table, but they are often used to send to the client strict portions (slices) of columns at proving time. Slices encode such portions by setting to 0 appropriate parts of the vector (which motivates one of the tests in our model and a new cryptographic property for ADS that we introduce, *zero testing*—see Fig. 8 and Section 5.2).

Once received the handles and the string π_{hint} that comprise the “proof” from the server, an idealized verifier is modeled as having special forms of oracle access to the set and slices handles (both from the prover and from indexing) as shown in Fig. 8). These allow

to: access subportions of slices (a *read*); combine slices *homomorphically* to define new ones; test whether the underlying objects satisfy some basic properties. Reading from a slice (through *read*) is one of the most important operations (e.g., in SELECT-type queries) and is the only one that does not return a bit or a handle. It is designed to model a *retrieval* operation that can be performed succinctly—the client receives a tuple data corresponding to the requested subset of indices X and nothing more¹³. The testing oracles available to the verifier allow it to check: (for set handles) union, containment, intersection, and complement relations; (for slice handles) whether the inner product of two slices equals a given value; (involving both slices and sets) whether a slice has value 0 on the positions indicated by a set handle. The last test—zero-testing—will later allow us to derive several complex operations from these basic ones.

Properties in idealized protocols: we require idealized VDBs to be complete (defined like in the cryptographic setting) and sound. The latter property informally requires that, for any supported query/database, there exists no set of handles that a malicious prover can “send” to make the client accept a wrong response. We also require an idealized VDB to be succinct, which roughly states that both the computational operations of the verifier and the total bandwidth remain approximately linear in the size of the query and its response. As we show in Section 5.2, a succinct idealized VDB can be compiled *directly* into a succinct cryptographic one. A full formalization of the idealized model is in Appendix B.

5.2 From Idealized to Concrete Protocols

Here we give an overview of how to compile idealized VDBs into cryptographic ones. For this, we will adopt (and extend) some of the ADS functionalities introduced in Section 4. Our general compilation strategy will follow the standard technique of associating to each “idealized object”—to each handle—a corresponding commitment (every time the idealized indexer or prover is expected to provide a handle, it will instead send a commitment to the underlying object). We will specifically commit to set handles through accumulators and to slice handles through vector commitments. We can then use the proving capabilities of these ADS to make concrete the verifier’s abstract interface defined in Fig. 8: each oracle query performed by the verifier will translate to a proof (and potentially a corresponding result) sent by the (cryptographic) prover (with the

¹²We will use them in qedb mainly for the latter purpose. A related “special” handle is O*, the set handle of *all row indices* (i.e., $\{1, \dots, \bar{n}\}$). See complement test in Fig. 8.

¹³Notice that, while we simplified our discussion talking about acceptance/rejection, a *read* is actually a mechanism through which an idealized verifier can also *produces* a response from the material sent by the prover. This is an arbitrary stylistic choice.

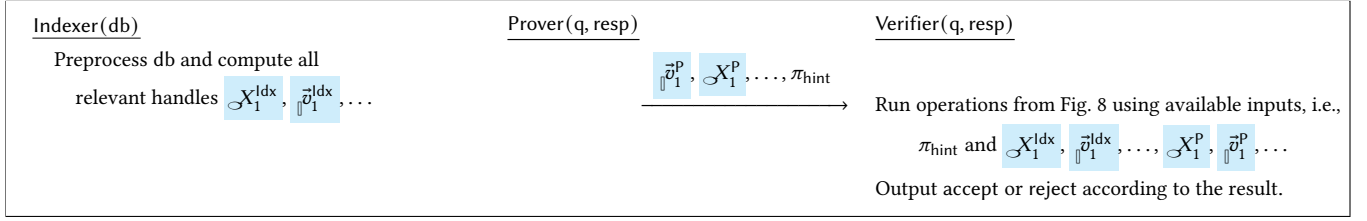


Figure 7: Interaction in our idealized protocols (simplified). Preprocessing is on the left; online interaction is on the right.

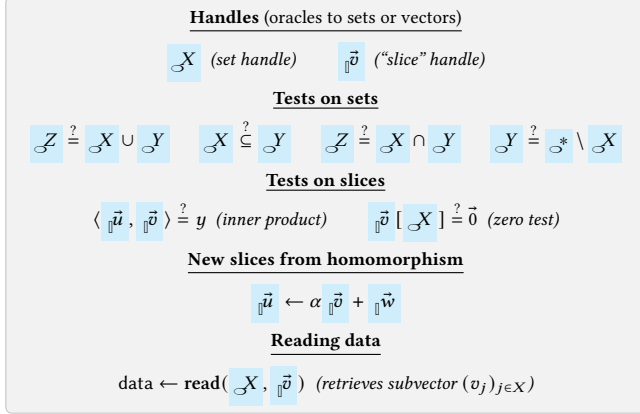


Figure 8: Verification oracles & operations in idealized VDBs.

only exception of the homomorphic computation of slices, since the verifier can perform it locally on its own).

The functionalities of homomorphic linear-map vector commitments (which can prove inner products of vectors), subvector openings (whose capability matches the **read** interface), and set accumulators (which can prove set relations) are an almost perfect match for our interface in Fig. 8. The only unsupported test is the ability to test (through handles) whether a slice is zero in a specified set of positions. We now discuss a new property of vector commitments and accumulators which provides exactly this.

Defining and instantiating our zero-testing functionality. We say that a vector commitment and an accumulator scheme support *zero-testing on accumulated sets* if, on input a vector commitment $\text{cm}_{\tilde{v}}$, an accumulator acc_X , and an appropriate proof, a verifier can be persuaded that $v_j = 0$ for all $j \in X$. We model security through an extractability property: any bounded adversary producing an accepting proof must “know” a vector $\tilde{v}$ and set X (in an appropriate domain of indices) consistent with the commitments such that $\tilde{v}$ truly vanishes on X .

We propose a very simple construction of zero-testing from polynomial commitments following standard techniques. It exploits the polynomial encoding underlying both primitives. The vector v is encoded as $p_v(X) = \sum_i v_i \lambda_i(X)$ via Lagrange interpolation, while the set S is encoded as its vanishing polynomial $Z_S(X) = \prod_{s \in S} (X - s)$. Observe that v is zero on all positions in S if and only if Z_S divides p_v —equivalently, there exists a quotient polynomial q such that $q(X) \cdot Z_S(X) = p_v(X)$. The prover commits to this quotient q , and the verifier checks the divisibility relation. To check this equation, we apply the Schwartz–Zippel lemma and let the verifier sample

a random challenge r ; the prover will then supply evaluations of the polynomials at r together with the corresponding proofs; the verifier will then check the proofs and that the polynomial identity holds. In Appendix D we fully formalize this property and provide an additional construction directly from pairing.

Our compilation results. Through the observations in this section and the results in Section 4 we are able to instantiate all the building blocks required by our compiler from pairings. Additionally, we prove in Appendix D that the two ADS with special properties we use for compilation (see Fig. 1) can generically be constructed from *any* extractable homomorphic polynomial commitment, which can directly lead to instantiations from different assumptions. Our main compilation result:

THEOREM 5.1 (INFORMAL STATEMENT OF THEOREMS D.1 AND D.2). *If there exists a succinct idealized VDB supporting query family Q , then there exists a succinct cryptographic VDB (Fig. 6) supporting Q under any of the following two conditions:*

- *there exists a homomorphic vector commitment and a set accumulator supporting set relations as in Fig. 1;*
- *there exists an extractable homomorphic polynomial commitment.*

Implication: post-quantum VDBs. An implication of our work and specifically of Theorem D.2 is that we can plug in *any extractable homomorphic polynomial commitment* and obtain a construction of verifiable databases. For example, using the polynomial commitment based on standard lattice assumptions from [53] to immediately obtain a plausibly post-quantum secure VDB construction¹⁴.

6 A New VDB Construction

Here we describe the main ingredients of our construction in terms of idealized sub-protocols. First, we discuss how the small set of core operations in Fig. 8 can *always* be used to obtain more “complex” operations, useful in VDBs (and potentially other settings). We will then give additional intuitions around how the idealized VDB for SQL behind qedb relies on the gadgets developed so far.

6.1 From Simple to Complex Operations in Idealized Protocols

In the previous section we defined a narrow idealized interface (Fig. 8). Despite being small, this interface is tantamount to a “RISC-like” set of primitive instructions from which one can derive more complex ones by composing native instructions. A (partial) list of the useful derived operations we can unconditionally obtain in our model is in Fig. 9 and a complete list is in Table 2 in Appendix C.

¹⁴The resulting construction has super-constant overhead for the verifier and proof size unlike the one in this work.

Some of the derived tests implied by the basic ones in Fig. 8:

$$\alpha \stackrel{?}{\leq} \llbracket \vec{v} \rrbracket \stackrel{?}{\leq} \beta \text{ (range check)} \quad \sum_{j \in X} \llbracket \vec{v}_j \rrbracket \stackrel{?}{=} y \text{ (sum check)}$$

$$X \stackrel{?}{=} \text{eqSet}(\llbracket \vec{u} \rrbracket, \llbracket \vec{v} \rrbracket) \text{ (slice equality test)}$$

Figure 9: Example of complex operations derived from the basic model in Fig. 8. A full list is in Table 2 in the Appendix.

Here we provide one example of a derivation. We will build specifically one of our most versatile derived tests, the “nullifying” test. Syntactically denoted as $\llbracket \vec{u} \rrbracket \stackrel{?}{=} \llbracket \vec{v} \rrbracket \left[X_0 \rightarrow 0 \right]$, it checks whether the slices $\vec{u}$ and $\vec{v}$ agree everywhere except on a given set X of positions, where $\vec{u}$ should be zero¹⁵. The way to construct it is very simple and works by applying the native zero-test (Fig. 8) twice and checking that: (i) $\vec{u}$ is zero in X (through the native zero test); and (ii) the difference $\vec{u} - \vec{v}$ is zero in the set of the remaining positions (the complement of X). The only operations besides the zero-testing include homomorphically computing slices and complement tests.

This test exemplifies two of the main techniques we use in developing these complex operations. One is the application of homomorphism as a way to test differences among slices. Another is using set complements to *partition* the space of possibilities (see (iii) and (iv)) to guarantee that a predicate is true.

6.2 A Specialized Construction for SQL (qedb)

Here we provide a glimpse of qedb in terms of idealized operations—both core and derived (Figures 8 and 9). Our final construction will simply from cryptographically compiling and optimizing our idealized blueprint. We showcase some of the mechanisms inside qedb through three query templates: conjunctive predicates, the aggregate function MIN, and JOINS. As background context for the descriptions below: in general, the preprocessing stage in qedb computes and stores slices for each column of the database (the preprocessing also computes some extra material depending on the query type; see full description in Appendix E).

Conjunctive query. Consider the following query pattern:

SELECT col_u FROM T WHERE $\text{Test}_1(\text{col}_v)$ AND $\text{Test}_2(\text{col}_w)$

Here the client is supposed to obtain (and validate) as a response a tuple of values $\vec{y}$ and a set $\hat{X} = \{i_1, \dots, i_\ell\}$ of rows in $\vec{u}$ (corresponding to col_u) with the guarantee that: $y_j = u_{i_j}$ for $j \in [\ell]$; and $\hat{X} = X_1 \cup X_2$ where, for $b \in \{1, 2\}$, $X_b := \{j : \text{Test}_b(j) = \text{true}\}$. To prove this query, we will simply rely on the expressivity of our idealized model (and of accumulators, under the hood). For the sake of clarity, we will assume that the verifier has received handles X_1 and X_2 about which it has already been persuaded that their content satisfy the predicate above (and we ignore the details of how the server has proven Test_b holds for them, since irrelevant here). Thanks to our framework, a protocol for conjunction is essentially trivial: the verifier (who has received $\hat{X}$ as part of the response) will simply query the intersection oracle to

¹⁵This test should not be confused with the native zero test, which differs semantically.

Pre-processing: Denote the set of positions of a value α in a vector v as the preimage set $\alpha^{-1}(v) := \{j : v[j] = \alpha\}$. The Data Owner defines the inverse lookup $X_p := \{\alpha^{-1}(\text{col}_p)\}_{\alpha \in V_p}$, where V_p is the set of values of column col_p . Then, it defines the set handle X_p .

Define analogously inverse lookup X_f for column col_f and its corresponding set handle X_f . Finally, the Data Owner defines the slice handles $\llbracket p \rrbracket$, $\llbracket f \rrbracket$ of columns col_p , col_f , as well as the slice handles of all the other columns of tables T_1 and T_2 .

Query’s answer verification:

- The prover sends:
 - $V_\cap = V_p \cap V_f$ (in the clear—this is not a handle).
 - it also sends for each $\alpha \in V_\cap$, the respective preimage set (as a handle) in each table. I.e., it sends $X_{\alpha, \text{col}}$ corresponding to $\alpha^{-1}(\text{col})$ for $\text{col} \in \{\text{col}_p, \text{col}_f\}$ and $\alpha \in V_\cap$.
- For each $\llbracket \text{col} \rrbracket \in \{\llbracket p \rrbracket, \llbracket f \rrbracket\}$ and $\alpha \in V_\cap$, the prover and the verifier run the preimage check sub-protocol $X_{\alpha, \text{col}} \stackrel{?}{=} \alpha^{-1}(\llbracket \text{col} \rrbracket)$.
- The prover and the verifier run the intersection sub-protocol to show $X_\cap \stackrel{?}{=} X_p \cap X_f$ (recall that the last two handles are already held by the verifier as handles from the preprocessing phase).
- Finally the verifier runs $\text{read}(\bigcup_\alpha X_{\alpha, p}, \llbracket T_1 \cdot \star \rrbracket)$ and $\text{read}(\bigcup_\alpha X_{\alpha, f}, \llbracket T_2 \cdot \star \rrbracket)$ (i.e., it reads from all the columns in T_1 and T_2 the rows that contain in columns col_p and col_f respectively the values from $V_\cap$).

Figure 10: The protocol for JOIN operations in qedb.

check $\hat{X} \stackrel{?}{=} X_1 \cup X_2$; it will then compute $\vec{y}$ itself by querying $\vec{y} \leftarrow \text{read}(\hat{X}, \llbracket \vec{v} \rrbracket)$. The final result, when compiled cryptographically, will result into a handful of pairings. This approach directly extends to OR and arbitrary logical predicates of this kind.

MIN query. Consider the query SELECT MIN(col) FROM T . Here the prover provides the set handle $X_{\min} = \{j_{\min}\}$, where $j_{\min}$ is an arbitrary position in the vector col containing $v_{\min}$. Having $X_{\min}$ and $\llbracket \text{col} \rrbracket$, the verifier retrieves the minimum value $v_{\min}$ from col (via read) and runs a subprotocol for range check (see Fig. 9) testing $\llbracket \text{col} \rrbracket - v_{\min} \cdot \vec{1} \stackrel{?}{\geq} \vec{0}$, to exclude that col contains a value smaller than $v_{\min}$.

JOINS. Recall that the result of a join between two tables consists of rows obtained by concatenating rows from each table that share equal values in the join columns. Typically, JOINS use, as join columns, a *primary key column* from the first table (unique, non-null values) and a corresponding *foreign key column* from the second table (values referencing that primary key, possibly repeated).

Consider two tables T_1, T_2 with respective columns col_p (primary key column of table T_1), and col_f (foreign key column in T_2 referencing values from col_p). Now consider the query:

SELECT * FROM T_1 JOIN T_2 ON $\text{col}_p = \text{col}_f$

To join two tables T_1 and T_2 on equality on columns, we run the protocol in Fig. 10. In that description we use sub-protocols defined in Appendix B (see Table 2). Such sub-protocols respectively perform a pre-image check (*what are all and only the rows containing a given value?*) and an intersection check.

The JOIN protocol works as follows: in the preprocessing phase, handles to columns of both tables are prepared. In particular, set handles for the join columns and slice handles for all other columns are formed. Both prover and verifier will use such public handles for building the proof and for verifying it. More precisely, the prover sends to the verifier the intersection set of values contained in both columns col_p and col_f , along with the set handles of such values, again for both columns col_p and col_f . Having such set handles and the slice handles of join columns, (i) the verifier may test—for every value in the intersection set and each join column—whether the all the occurrences of the value in a column are listed in the corresponding inverted index (pre-image sub-protocol); (ii) the verifier checks whether all the values in the intersection set sent by the prover are effectively contained in the intersection of the values contained in the join columns (intersection sub-protocol). Finally, the verifier retrieves the rows from T_1 and T_2 containing the values in the intersection set and computes their cartesian product, obtaining the final result on the JOIN operation. The protocol above directly extends to multi-way JOINS.

REMARK 1 (UPDATES IN qedb). *So far we have described our framework considering a static database. Of course, many applications require updating the data and hence the corresponding vector commitments and accumulators. Fortunately, it is not necessary to recompute them from scratch, as data changes. This is because of the homomorphic properties of the LVC scheme. In particular, see Section 4.4 of [22]. Updating the accumulators in the preprocessing cannot be done in constant-time in a publicly verifiable setting, but our experiments show it to be practical for several common updates.*

7 Experimental Evaluation and Comparison

In this section, we first describe our implementation and show the performance of qedb on representative queries. We then compare qedb to other approaches.

7.1 Evaluation of qedb

We have implemented our construction using the Rust programming language and the Arkworks library [34] for efficient cryptographic operations. We use the BLS12-381 as a pairing-friendly elliptic curve. We run our experiments in a multi-threaded setting on a commodity laptop: a MacBook Pro featuring 11 cores and 36 GB of RAM¹⁶.

Our evaluation aimed at answering the questions:

- (1) What are the concrete running times and communication requirements of qedb in a representative setting?
- (2) What is the effect of database sizes on running times¹⁷?

In the remainder of this subsection, we present the experimental results obtained by evaluating our system to answer the two questions above. We then compare our approach with other existing VDB designs—IntegriDB, sumcheck-based schemes, and zkVM-based schemes—in Sections 7.2, 7.3, and 7.4, respectively.

¹⁶Note that our prover has very low memory requirements: less than 5 GB of RAM to handle tables of 2^{17} rows. NB: we use multi-threading only for stand-alone evaluations; we compare to other schemes using single-threading.

¹⁷Proof size is not an interesting metric here—it is independent of DB size in qedb.

Experimental queries and dataset. We performed our experiments using the queries described in Section 1.2 (see also Fig. 2). These queries showcase some of the expressivity features of our system. Overall, they involve different types of queries—aggregate queries (e.g., SUM and COUNT) as well as SELECT—and different types of filtering conditions—conjunctions of predicates, range queries, simple equality tests on target values, comparison among columns. We remark that empirical SQL workload studies show that the vast majority of the queries are structurally simple, involving a small number of filtering conditions and aggregations, over a limited number of tables [70, 92]. The test database consists of a single table with columns populated through synthetic data (generated by sampling random values in the appropriate domain and converting them into scalars). Our main experiments in this section will be on a table with 2^{17} rows (the SELECT in Q_{Match} has a response of $\sim 1\text{K}$ rows). This number is chosen both because it is representative of a medium-sized DB and because it roughly corresponds to the cut-off point where previous state-of-the-art solutions (IntegriDB [96]) experimentally fails to be able to perform proving due to its memory usage¹⁸. In contrast, qedb can run on DBs of millions of rows.

Our experiments on a representative DB size are presented in Table 1; those testing proving scalability are in Fig. 11.

REMARK 2. *In our benchmarks, the client has direct access to the components of the digest (vector commitments to columns, etc.). The reason for this choice is twofold: it has little impact on performance (it is amortized after a few queries), and it makes substantially simpler to explain the sources of our costs in the remainder of this section.*

Proof Size. The proof sizes for Q_{Tot} and Q_{Match} reflect a typical ballpark for the proof sizes in qedb, i.e., ≈ 1 KB. These two are distinct types of queries, respectively a SUM and a SELECT. The larger proof in Q_{CntTx} is due to our range query; this can be reduced by at least 25% through simple optimizations. As mentioned, our proof size is independent of the sizes of the response and the DB.

Verification time. Our verifier runs in constant time for queries Q_{Tot} and Q_{CntTx} , specifically ≈ 6 ms for Q_{Tot} and ≈ 13 ms for Q_{CntTx} , independently of the database size. The verification time for Q_{Match} does not grow despite the larger proof size because it mostly consists of pairing equations, which can easily be checked in batch. The only query that admits different response sizes—and where, as a consequence, the verifier may need to perform more work—is Q_{Match} . We tested this query on larger responses and observed that increasing its size by a factor of $5\times$ results in only a modest increase in verification time, from ≈ 17 ms to approximately 23 ms ($\approx 1.35\times$).

Proving time. Our prover is able to run all three queries in seconds, with Q_{CntTx} taking the longest—around 15 s on a database of size 2^{17} —due to the cost of range proofs. From our additional experiments, the prover scales quasilinearly with the size of the database for aggregation queries such as Q_{Tot} and Q_{CntTx} . For example, on a database of size 2^{20} (about $8\times$ larger than in Table 1), both queries run in roughly $8\text{--}10\times$ the time reported for the smaller database. For SELECT queries such as Q_{Match} —in which the prover has some dependency on the response size—increasing the database size also exhibits quasilinear behavior.

¹⁸This was tested on a 128GB RAM machine; the IntegriDB paper also documents this phenomenon with similar numbers.

Query	Prover Time	Verifier Time	Proof Size
Q_{Tot}	3.29 s	6.89 ms	0.89 KB
Q_{CntTx}	15.22 s	13.71 ms	4.86 KB
Q_{Match}	1.49 s	17.58 ms	0.97 KB

Table 1: Experimental evaluation over queries from Fig. 2 on a DB with 2^{17} rows.

Pre-processing of the database. We now report on the time required to generate the cryptographic material during the DB pre-processing (not in Table 1) and the disk space required to store it. In general, both metrics grow linearly with the number of rows and columns in the database. For a table with 2^{17} rows, generation takes approximately 2.6 s for 5 columns and 4.3 s for 9 columns, requiring 75 MB and 134 MB of storage, respectively. For a larger dataset with 2^{20} rows, the generation time increases to about 33 s for 5 columns and 60 s for 9 columns, with storage requirements of 523 MB and 890 MB, respectively.

7.2 IntegriDB vs qedb

Below, we discuss how our scheme compares to IntegriDB (the state of the art on ADS-based VDB). We remind the reader about some of the already discussed metrics in which we fare better than IntegriDB: a more expressive proof system, storage linear in the size of the database (instead of quadratic in the number of columns), verification and proof size independent of database size. Additionally, we remark that, even on databases of moderate sizes ($\approx 10^5$ rows per table with a total of ≈ 50 columns across all tables), the preprocessing time and storage requirements of IntegriDB may seem hard to justify in concrete practical settings. Indeed, from conservative estimates extrapolating their reported data for smaller tables we predict around ≈ 3 days required for preprocessing and a server storage of approximately 500GB ($\approx 300\times$ the requirements to store the database alone). For these reasons, here we simply provide a few additional high-level points of comparison performance-wise.

Pre-processing. The asymptotic preprocessing time improvements of qedb over IntegriDB are confirmed experimentally. Even on small tables (5 columns with ≈ 1000 rows each) we observe roughly two orders of magnitude speedups in qedb’s preprocessing compared to IntegriDB; this ratio grows further as the database size increases.

Proving Time. We find that, even on medium-sized tables ($\geq 10^5$ rows), IntegriDB’s prover could not run due to memory overhead (see also Footnote 18). Our estimates show that qedb has competitive performance compared to IntegriDB across queries and table sizes (but with substantially better proof size): both schemes, run single-threaded, have proving times of approximately ≈ 100 s on representative queries and moderately-sized tables.

Proof Size. Our proof sizes show an order of magnitude improvement over IntegriDB on the same queries. For common queries with simple equality checks, our proofs are around 1 KB, independently of the number of rows; IntegriDB’s proofs grow with table size and are about 5 KB even for small tables¹⁹. Other query types yield worse proof sizes; even for range queries our proof size is about 5

KB, compared to roughly 20 KB for single-dimensional range proofs in IntegriDB.

Verification Time. In contrast to IntegriDB, our verifier runs in constant time (of the order of milliseconds independently of the database size). We observe at least one class of settings in which this leads to noticeable concrete savings for qedb: for any query involving range checks—where IntegriDB requires $\approx 2 \log(|\text{db}|)$ pairings—we observe that qedb’s verification (run single-threaded) is better than IntegriDB’s for any table larger than 1000 rows; we estimate a speedup of approximately $1.7\times$ starting from 2^{16} rows.

7.3 Sumcheck-based schemes vs qedb

Here we compare to techniques based on multivariate sumchecks and GKR-style protocols [46]. This family is a relevant point of comparison for two reasons: it includes vSQL [95], the first effort to obtain fast VDBs from more general-purpose approaches; it underpins some of the fastest proof systems available (see discussion in [87]). See Appendix H for additional technical background. Compared to sumcheck- and GKR-based protocols, our scheme offers stronger succinctness and simpler security. Its proofs are constant-sized, while GKR-based grow logarithmically with database size—already $2\text{--}3\times$ larger on moderately large datasets. Verification costs are similar, but qedb benefits for precomputation for fast proving, unlike GKR-based protocols which incur into overhead from interactive rounds and circuit generation. Moreover, our scheme applies Fiat-Shamir only once, yielding simpler, more robust security than GKR-based multi-round approach, which suffers from weaker security guarantees.

More in detail, we compare against a protocol template which can be considered a more fine-grained version of vSQL (where we run not on the whole database, but only on the columns we need), at the high level, works as follows: at preprocessing time, we encode each column in the database as a multivariate polynomial commitment that can be plugged into a GKR-style protocol [46]; in order to prove the result of a query, server and client run an instance of the GKR protocol for the specific query. One could make other choices for such a protocol template—to which we refer hereby as Π_{GKR} —but we focus on those giving strong succinctness features and fast proving. A reasonable instantiation for the polynomial commitment is HyperKZG, the variant of KZG for multivariate polynomials proposed in [7]. This choice is motivated by its efficiency, popularity (it was used, e.g., [28] among others) and because it uses the same trusted setup as our setting.

Proof size and verification. In Π_{GKR} , the final proof consists of *i*) a GKR/sumcheck transcript and *ii*) proofs of opening for the underlying polynomial commitment; the communication complexity of these two items is respectively polylogarithmic (depending on the query) and logarithmic in the number of rows of the tables involved. In contrast, qedb has $O(1)$ proof size. We can provide a concrete comparison for the proof size costs between Π_{GKR} and qedb through a conservative estimate. Observe that, on *any* query, the proof size of Π_{GKR} is at least $\log(|\text{#rows}|)(2|\mathbb{F}| + |\mathbb{G}_1|)^{20}$. This implies that, even on a DB of moderate size like the one in Table 1, the Π_{GKR} proofs are $2\text{--}3\times$ larger than qedb for queries like Q_{Tot} and Q_{Match} . We stress that this is a very conservative estimate and the

¹⁹Using IntegriDB’s code [55]: SELECT queries with value equality on tables of about 1000 rows produce proofs of 5 KB (variable with value distribution); SUM queries on similar tables produce proofs of 10 KB.

²⁰This is the bare minimum for a sumcheck interaction and one HyperKZG opening.

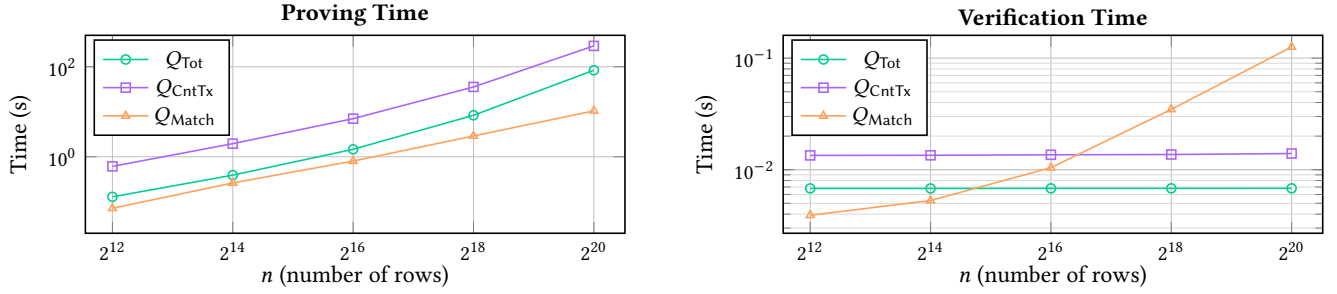


Figure 11: Performance of the prover (P) and verifier (V) for the Fig. 2 as a function of the number of rows n (log-log scale).

overhead in Π_{GKR} is likely to be larger; also, it will become worse as $|\text{db}|$ increases. For what concerns verification time, we estimate the performance of Π_{GKR} to be comparable to ours: Π_{GKR} ’s verification cost consists of executing $O(1)$ pairings and a logarithmic number of $\mathbb{G}_1$ and field operations.

Proving time. We discuss to which extent the prover performance of the two schemes is similar and when they are likely to diverge. We start by looking at the case of very simple queries, where Π_{GKR} is likely to have comparable costs to those of qedb: in this case—where the GKR circuits are plausibly small—the costs in both schemes are dominated by the (same number of) openings of polynomial commitment for Π_{GKR} and in the linear-map vector commitment for qedb; these openings are dominated by $\approx |\text{\#rows}| \mathbb{G}_1$ operations in both schemes. On the other hand, Π_{GKR} is likely to have extra overhead in terms of field operations to generate the GKR transcript even in relatively simple queries due to the complexity of the resulting circuit. Consider a core query operation such as set intersection. When expressed as a circuit, its most straightforward implementation is not only inefficient—it involves a quadratic number of steps—but also unnecessarily complicated. More efficient approaches add even more complexity, involving sorting networks and/or handling duplicates (see vSQL paper, Sec. VI.C [95]).

Complexity and security. Π_{GKR} requires expressing queries as circuits in contrast to qedb. We remark, however, that for *some* elementary queries (e.g., SELECT, SUM or COUNT *without any* AND/OR) Π_{GKR} can offer a relatively simple protocol since the related circuits are simple formulas; this is not true of all queries though—see previous point about implementing set intersection. More generally, in contrast to qedb, Π_{GKR} requires writing an explicit circuits for even small variations on query types (or pay the overhead for a “semi-universal” query circuit). The security of Π_{GKR} , like in the case of qedb boils down almost exclusively to that of its polynomial commitment. The main difference in assumptions and security analysis between Π_{GKR} and qedb lies in how they use the Fiat-Shamir heuristic. In fact, qedb applies it to a *single* round of prover-verifier interaction and exclusively in its instantiations (the idealized protocol has no interaction). In contrast, depending on the query, Π_{GKR} must apply it to a logarithmic (or polylogarithmic) number of rounds. This leads to a more complex analysis and substantially worse security loss (see, e.g., Corollary 5.7 in [23] for the relation between number of rounds and security of Fiat-Shamir). More crucially, Fiat-Shamir in the GKR world has been recently shown to be, in general, fundamentally insecure [58].

7.4 zkVM-based schemes vs qedb

A popular general-purpose design for proving systems is that of zkVMs²¹, whose distinguishing feature is to move away from circuits and constraint systems to encode a computation²². Instead, zkVMs work by describing some predetermined CPU abstraction and by proving the execution of a computer program expressed in a corresponding instruction set. Typically, zkVM proving works by dividing the computation trace into segments, then applying a tree of recursive proving where each leaf is a segment. After this process, the final (relatively large) proof is then compressed through a scheme that produces very short certificates, such as [47].

Performance and expressivity. When run on computers with many cores (or clusters of GPUs) with hundreds of GBs of RAM, zkVMs show highly performant provers (see, e.g., [89]). We are unaware of discussions or evaluations on the performance of zkVMs on more modest hardware, where our benchmarks suggest qedb has very promising performance. The verifier in a zkVM consists typically of a few pairing operations (e.g., if the final prover is instantiated with Plonk [45]); thus, verification time should generally be comparable to that in qedb and will depend on the specific instantiation of the final proving step. Regarding expressivity, in principle a VM-based approach allows one to describe any SQL query; in settings where one requires more exotic features of SQL, zkVMs may be an appealing choice.

Complexity and security. zkVMs are substantially more complex than qedb and other ADS-based solutions. By design, they incorporate many moving parts: instruction decoding, memory consistency checks, trace segmentation, recursive aggregation, and final proof compression (see Appendix I for further discussion). This architectural complexity makes zkVMs harder to audit and maintain, and potentially more prone to implementation bugs. As a rough quantitative indicator, we observe that typical zkVM implementations contain over an order of magnitude more lines of code than qedb—and this comparison does not even account for the SQL-specific logic that would need to be built on top. In settings where the full expressivity of SQL is not required, qedb may thus offer a more

²¹This is a popular phrase to refer to this type of schemes, but it is a misnomer. Despite the name, all but very few of the popular “zk”VM designs lack zero-knowledge (exceptions are [13, 54]). That is, almost none of these constructions offers confidentiality.

²²A constraint system is yet another way to encode a computation for the purpose of proving it through a cryptographic argument. It offers some flexibility compared to circuits [80], but it inherits some of the drawbacks of proving circuit execution. These include having to express the program in a relatively cumbersome representation [79] and (generally) requiring the proving algorithm to store the entire trace of the computation in memory.

prudent choice from a software-engineering standpoint. From a provable-security perspective, zkVMs present additional concerns. To our knowledge, no zkVM construction comes with a complete, end-to-end security proof; the sheer scale of these systems makes such an analysis impractical. More subtly, even individual components often rest on assumptions that are heuristic, not well-founded, or not yet well-understood. We highlight a few such concerns in Appendix J—not to suggest that zkVMs are insecure, but to emphasize that their security posture demands considerably more caution than that of simpler constructions like qedb—discussing: aggressive coding-theoretic conjectures, the heuristic security of recursive proving and concerns related to Fiat-Shamir.

8 Conclusion and Future Work

We have presented qedb, a new construction for VDBs that advances the state of the art. Research in this direction had remained largely dormant since IntegriDB [96], with subsequent efforts gravitating toward general-purpose proof systems. Our work shows that the ADS paradigm still has considerable untapped potential: by combining modern ADS with a modular framework, qedb achieves better asymptotic tradeoffs, broader expressivity, and a simpler architecture—while maintaining provable-security guarantees.

Future work. The design of qedb opens several avenues for future research. One first natural direction is *formal verification*: because our construction relies on a small set of well-defined cryptographic building blocks with clean interfaces, it lends itself to machine-checked proofs of correctness for its implementation. We view this as a promising path toward high-assurance deployments and we have ongoing work on this.

Another direction concerns the application of *lookup arguments* [11, 14, 15, 44, 51, 84, 93], which appear conceptually close to some operations in our idealized model. Whether lookups can yield practical variants of our protocols remains to be seen: they are optimized for multisets (which may be unnecessary for SQL) and do not perfectly match our interface, but the connection merits further exploration.

Adding *zero-knowledge* to ADS-based VDBs in a simple manner is another open problem. Switching to hiding polynomial commitments is a natural starting point, though additional leakage channels in the protocol would need to be addressed. Techniques from [93] and the Curve Trees/Forests line of work [20, 21] offer potential approaches in the transparent setting under DLOG.

Finally, we pose a foundational question: *are there inherent limits to the succinctness–expressivity tradeoffs achievable by ADS-based approaches?* That is, can one prove lower bounds on communication complexity for expressive queries? The tight correspondence between ADS-based verification and data-structure access patterns suggests that results from the data-structures literature—both authenticated and classical—may offer insight.

Acknowledgments and Disclosure

Matteo Campanelli would like to thank Mahak Pancholi, who provided useful feedback on this manuscript, and Nicola Greco who, around 2022, first suggested verifiable databases as an interesting topic to work on. Matteo has no financial interest in Provably Technologies and has pursued this research work on a voluntary basis,

motivated by the potential relevance to the cryptographic proofs community.

References

- [1] a16z Contributors. 2024. Jolt: A zero-knowledge virtual machine. <https://github.com/a16z/jolt>.
- [2] Diego F Aranha, Emil Madsen Bendsen, Matteo Campanelli, Chaya Ganesh, Claudio Orlandi, and Akira Takahashi. 2022. ECLIPSE: Enhanced compiling method for pedersen-committed zkSNARK engines. In *IACR International Conference on Public-Key Cryptography*. Springer, 584–614.
- [3] Axiom. 2025. Axiom OpenVM. <https://www.axiom.xyz/>.
- [4] Rahul Batra. 2018. SQL primer. *Apress, Berkeley, CA* (2018), 183–187.
- [5] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. 2016. Interactive oracle proofs. In *Theory of Cryptography Conference*. Springer, 31–60.
- [6] Josh Benaloh and Michael de Mare. 1994. One-Way Accumulators: A Decentralized Alternative to Digital Signatures. In *Advances in Cryptology – EUROCRYPT ’93*, Tor Helleseth (Ed.).
- [7] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orrù. 2022. Gemini: Elastic SNARKs for Diverse Environments. In *Advances in Cryptology – EUROCRYPT*.
- [8] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. 2020. Transparent SNARKs from DARK compilers. In *Advances in Cryptology – EUROCRYPT 2020*.
- [9] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. 2019. Proofs for Inner Pairing Products and Applications. *Cryptology ePrint Archive*, Paper 2019/1177. <https://eprint.iacr.org/2019/1177>
- [10] Matteo Campanelli, Felix Engelmänn, and Claudio Orlandi. 2022. Zero-knowledge for homomorphic key-value commitments with applications to privacy-preserving ledgers. In *International Conference on Security and Cryptography for Networks*. Springer.
- [11] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. 2024. Lookup Arguments: Improvements, Extensions and Applications to Zero-Knowledge Decision Trees. In *Public-Key Cryptography – PKC*.
- [12] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. 2021. Lunar: A Toolbox for More Efficient Universal and Updatable zkSNARKs and Commit-and-Prove Extensions. In *Advances in Cryptology – ASIACRYPT*.
- [13] Matteo Campanelli, Antonio Faonio, and Luigi Russo. 2025. SNARKs for virtual machines are non-malleable. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 153–183.
- [14] Matteo Campanelli, Dario Fiore, and Rosario Gennaro. 2025. Natively compatible super-efficient lookup arguments and how to apply them. *Journal of Cryptology* 38, 1 (2025), 14.
- [15] Matteo Campanelli, Dario Fiore, Semin Han, Jihye Kim, Dimitris Kolonelos, and Hyunok Oh. 2022. Succinct zero-knowledge batch proofs for set accumulators. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 455–469.
- [16] Matteo Campanelli, Dario Fiore, and Hamidreza Khoshakhlagh. 2024. Witness Encryption for Succinct Functional Commitments and Applications. In *Public-Key Cryptography – PKC*.
- [17] Matteo Campanelli, Dario Fiore, and Mahak Pancholi. 2025. On Composing AGM-Secure Functionalities with Cryptographic Proofs: Applications to Unbounded-Depth IVC and More. *Cryptology ePrint Archive*, Paper 2025/2086. <https://eprint.iacr.org/2025/2086>
- [18] Matteo Campanelli, Dario Fiore, and Mahak Pancholi. 2025. When Can We Incrementally Prove Computations of Arbitrary Depth? *Cryptology ePrint Archive*, Paper 2025/1413. <https://eprint.iacr.org/2025/1413>
- [19] Matteo Campanelli and Mathias Hall-Andersen. 2024. Fully Succinct Arguments over the Integers from First Principles. *Cryptology ePrint Archive*, Paper 2024/1548. <https://eprint.iacr.org/2024/1548>
- [20] Matteo Campanelli, Mathias Hall-Andersen, and Simon Holmgård Kamp. 2023. Curve trees: Practical and transparent {Zero-Knowledge} accumulators. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4391–4408.
- [21] Matteo Campanelli, Mathias Hall-Andersen, and Simon Holmgård Kamp. 2024. Curve forests: Transparent zero-knowledge set membership with batching and strong security. *Cryptology ePrint Archive* (2024).
- [22] Matteo Campanelli, Anca Nitulescu, Carla Ràfols, Alexandros Zacharakis, and Arantxa Zapico. 2022. Linear-Map Vector Commitments and Their Practical Applications. In *Advances in Cryptology – ASIACRYPT*.
- [23] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. 2018. Fiat-Shamir From Simpler Assumptions. *Cryptology ePrint Archive*, Paper 2018/1004. <https://eprint.iacr.org/2018/1004>
- [24] Dario Catalano, Mario Di Raimondo, Dario Fiore, and Mariagrazia Messina. 2011. Zero-Knowledge Sets With Short Proofs. *IEEE Transactions on Information Theory* 57, 4 (2011), 2488–2502.
- [25] Dario Catalano and Dario Fiore. 2013. Vector Commitments and Their Applications. In *Public-Key Cryptography – PKC*.

- [26] Dario Catalano, Dario Fiore, and Ida Tucker. 2022. Additive-Homomorphic Functional Commitments and Applications to Homomorphic Signatures. In *Advances in Cryptology - ASIACRYPT*.
- [27] Melissa Chase, Alexander Healy, Anna Lysyanskaya, Tal Malkin, and Leonid Reyzin. 2005. Mercurial Commitments with Applications to Zero-Knowledge Sets. In *Advances in Cryptology - EUROCRYPT*, Ronald Cramer (Ed.). Springer.
- [28] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *Advances in Cryptology - EUROCRYPT*.
- [29] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. 2023. Security Bounds for Proof-Carrying Data from Straightline Extractors. *Cryptology ePrint Archive*, Paper 2023/1646. <https://eprint.iacr.org/2023/1646>
- [30] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas Ward. 2019. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. *Cryptology ePrint Archive*, Paper 2019/1047. <https://eprint.iacr.org/2019/1047>
- [31] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. In *Advances in Cryptology - EUROCRYPT*.
- [32] Istehad Chowdhury and Mohammad Zulkernine. 2011. Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities. *Journal of Systems Architecture* 57, 3 (2011), 294–313.
- [33] European Commission. 2025. Data Act Explained. <https://digital-strategy.ec.europa.eu/en/factpages/data-act-explained>
- [34] Arkworks contributors. 2022. arkworks zkSNARK ecosystem. <https://arkworks.rs>
- [35] Elizabeth Crites and Alistair Stewart. 2025. On Reed–Solomon Proximity Gaps Conjectures. *Cryptology ePrint Archive*, Paper 2025/2046. <https://eprint.iacr.org/2025/2046>
- [36] Ivan Damgård. 1991. Towards practical public key systems secure against chosen ciphertext attacks. In *Advances in Cryptology - CRYPTO '91*. Springer, 445–456.
- [37] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. 2023. Weak fiat-shamir attacks on modern proof systems. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 199–216.
- [38] Leo de Castro and Chris Peikert. 2023. Functional Commitments for All Functions, with Transparent Setup and from SIS. In *Advances in Cryptology - EUROCRYPT*, Carmi Hazay and Martijn Stam (Eds.).
- [39] Deloitte. 2023. It's time to reprioritize BCBS239 compliance. <https://www.deloitte.com/us/en/services/consulting/articles/basel-risk-data-aggregation-and-reporting-requirements.html>
- [40] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. 2020. Line-Point Zero Knowledge and Its Applications. *IACR Cryptol. ePrint Arch.* (2020), 1446.
- [41] Bank for International Settlements. 2013. Principles for effective riskdata aggregation and risk reporting. BIS report BCBS239.
- [42] World Economic Forum. 2025. Data Integrity (Blockchain Toolkit). <https://widgets.weforum.org/blockchain-toolkit/data-integrity/index.html>
- [43] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. 2018. The Algebraic Group Model and its Applications. In *Advances in Cryptology - CRYPTO*.
- [44] Ariel Gabizon and Zachary J. Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive*, Paper 2020/315. <https://eprint.iacr.org/2020/315>
- [45] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *IACR Cryptol. ePrint Arch.* (2019), 953.
- [46] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2015. Delegating Computation: Interactive Proofs for Muggles. *J. ACM* 62, 4 (2015), 27:1–27:64.
- [47] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology - EUROCRYPT*.
- [48] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proc. ACM Manag. Data* 3, 1 (2025), 63:1–63:27.
- [49] Fuchun Guo, Yi Mu, and Zhide Chen. 2007. Identity-based encryption: How to decrypt multiple ciphertexts using a single decryption key. In *International Conference on Pairing-Based Cryptography*. Springer, 392–406.
- [50] Hacken. 2025. <https://hacken.io/discover/blockchain-oracles/>. <https://hacken.io/discover/blockchain-oracles/>
- [51] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. 2025. SoK: Lookup Table Arguments. *Cryptology ePrint Archive*, Paper 2025/1876. <https://eprint.iacr.org/2025/1876>
- [52] HSBC. 2025. Form 20-F. <https://www.hsbc.com/-/files/hsbc/investors/hsbc-results/2024/annual/pdfs/hsbc-holdings-plc/250220-hsbc-holdings-plc-form-20-f.pdf>
- [53] Intak Hwang, Jinyeong Seo, and Yongsoo Song. 2024. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *Advances in Cryptology - CRYPTO*. Springer, 414–448.
- [54] Liger Inc. 2025. Liger. <https://liger-inc.com/>.
- [55] IntegriDB. 2015. IntegriDB Code Repository. <https://github.com/integridb/Code>.
- [56] JPMorganChase. 2024. Form 10-K. <https://www.jpmorganchase.com/content/dam/jpmc/jpmorgan-chase-and-co/investor-relations/documents/quarterly-earnings/2024/4th-quarter/corp-10k-2024.pdf>
- [57] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology - ASIACRYPT*.
- [58] Dmitry Khovratovich, Ron D Rothblum, and Lev Soukhanov. 2025. How to prove false statements: practical attacks on Fiat-Shamir. In *Advances in Cryptology - CRYPTO '25*. Springer, 3–26.
- [59] Lagrange Labs. 2025. ZK Co-processor. <https://www.lagrange.dev/zk-coprocessor>.
- [60] Russell W. F. Lai and Giulio Malavolta. 2019. Subvector Commitments with Application to Succinct Arguments. In *Advances in Cryptology - CRYPTO*.
- [61] Hyeonbum Lee and Jae Hong Seo. 2024. On the Security of Nova Recursive Proof System. *Cryptology ePrint Archive*, Paper 2024/232. <https://eprint.iacr.org/2024/232>
- [62] Feifei Li, Marios Hadjieleftheriou, George Kollios, and Leonid Reyzin. 2006. Dynamic authenticated index structures for outsourced databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, 121–132.
- [63] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proc. VLDB Endow.* 16, 8 (2023).
- [64] Benoît Libert, Khoa Nguyen, Benjamin Hong Meng Tan, and Huaxiong Wang. 2019. Zero-Knowledge Elementary Databases with More Expressive Queries. In *Public-Key Cryptography - PKC*, Dongdai Lin and Kazuo Sako (Eds.). Springer.
- [65] Benoît Libert, Somindu C. Ramanna, and Moti Yung. 2016. Functional Commitment Schemes: From Polynomial Commitments to Pairing-Based Accumulators from Simple Assumptions. In *43rd International Colloquium on Automata, Languages, and Programming, ICALP*.
- [66] Benoît Libert and Moti Yung. 2010. Concise Mercurial Vector Commitments and Independent Zero-Knowledge Sets with Short Proofs. In *Theory of Cryptography, 7th Theory of Cryptography Conference, TCC*.
- [67] Mary Maller, Nicolas Mohnblatt, and Arantxa Zapico. 2025. IVC in the Open-and-sign Random Oracle Model. *Cryptology ePrint Archive*, Paper 2025/1663. <https://eprint.iacr.org/2025/1663>
- [68] Silvio Micali, Michael O. Rabin, and Joe Kilian. 2003. Zero-Knowledge Sets. In *44th Symposium on Foundations of Computer Science (FOCS 2003)*. IEEE Computer Society.
- [69] Andrew Miller, Michael Hicks, Jonathan Katz, and Elaine Shi. 2014. Authenticated data structures, generically. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL*. ACM, 411–424.
- [70] Clement Moreau, Clement Legroux, Verónica Peralta, and Mohamed Ali Hamrouni. 2022. Mining SQL workloads for learning analysis behavior. *Inf. Syst.* 108 (2022).
- [71] Einar Mykletun, Maithili Narasimha, and Gene Tsudik. 2006. Authentication and integrity in outsourced databases. *ACM Trans. Storage* 2, 2 (2006), 107–138.
- [72] Lan Nguyen. 2005. Accumulators from Bilinear Pairings and Applications. In *Topics in Cryptology - CT-RSA*. Springer, 275–292.
- [73] US Treasury Office of Financial Research. 2016. Developing Best Practices for Regulatory Data Collections. https://www.financialresearch.gov/viewpoint-papers/files/OFRvp-2016-01_Best-Practices-Data-Collection.pdf
- [74] Bank of International Settlements. 2023. The Oracle Problem and the Future of DeFi. <https://www.bis.org/publ/bisbull76.pdf>
- [75] Rafail Ostrovsky, Charles Rackoff, and Adam D. Smith. 2004. Efficient Consistency Proofs for Generalized Queries on a Committed Database. In *Automata, Languages and Programming: 31st International Colloquium, ICALP 2004 (Lecture Notes in Computer Science, Vol. 3142)*. Springer, 1041–1053.
- [76] Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2011. Optimal Verification of Operations on Dynamic Sets. In *Advances in Cryptology - CRYPTO 2011*.
- [77] Plonky3. 2024. Plonky3. <https://github.com/Plonky3/Plonky3>.
- [78] Gholamreza Ramezan, Eladio Robles Casas, Ben Beath, and Jake Godfrey. [n. d.]. zk-Database: Privacy-enabled Databases using Zero-Knowledge Proof. In *Proceedings of the 2024 7th International Conference on Blockchain Technology and Applications, ICBTA, publisher = ACM, year = 2024*.
- [79] Michael Zhu Sam Ragsdale and Justin Thaler. 2023. Understanding Lasso and Jolt, from theory to code. <https://a16zcrypto.com/posts/article/building-on-lasso-and-jolt/>.
- [80] Srinath Setty, Justin Thaler, and Riad Wahby. 2023. Customizable constraint systems for succinct arguments. *Cryptology ePrint Archive*, Paper 2023/552. <https://eprint.iacr.org/2023/552>
- [81] Yonghee Shin, Andrew Meneely, Laurie Williams, and Jason A Osborne. 2010. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE transactions on software engineering* 37, 6 (2010), 772–787.
- [82] Victor Shoup. 1997. Lower bounds for discrete logarithms and related problems. In *International Conference on the Theory and Applications of Cryptographic*

Techniques. Springer, 256–266.

- [83] Donghyun Sohn, Xiling Li, and Jennie Rogers. 2024. Everything You Always Wanted to Know About Secure and Private Database Systems (but were Afraid to Ask). *IEEE Data Eng. Bull.* 47, 2 (2024).
- [84] Tomer Solberg. 2023. A Brief History of Lookup Arguments. <https://github.com/ingonyama-zk/papers/blob/main/lookups.pdf>.
- [85] Space and Time. 2025. Proof-of-SQL Co-processor. <https://github.com/spaceandtimefdn/sxt-proof-of-sql>.
- [86] Succinct Labs. 2024. SP1: A zero-knowledge virtual machine. <https://github.com/succinctlabs/sp1>.
- [87] Justin Thaler. 2025. Sum-check Is All You Need: An Opinionated Survey on Fast Provers in SNARK Design. Cryptology ePrint Archive, Paper 2025/2041. <https://eprint.iacr.org/2025/2041>
- [88] Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist, and Dmitry Khovratovich. 2020. Aggregatable subvector commitments for stateless cryptocurrencies. In *International Conference on Security and Cryptography for Networks*. Springer, 45–64.
- [89] V.A. 2025. zkVM Benchmarks. <https://zkbenchmarks.com/>.
- [90] Hoeteck Wee and David J. Wu. [n. d.]. Succinct Functional Commitments for Circuits from k-Lin. In *Advances in Cryptology - EUROCRYPT, year = 2024*.
- [91] Hoeteck Wee and David J. Wu. 2023. Succinct Vector, Polynomial, and Functional Commitments from Lattices. In *Advances in Cryptology - EUROCRYPT*.
- [92] Ting Xie, Varun Chandola, and Oliver Kennedy. 2018. Query Log Compression for Workload Analytics. *Proc. VLDB Endow.* 12, 3 (2018), 183–196.
- [93] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. 2022. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3121–3134.
- [94] Zcash. 2025. The Halo2 Book. <https://github.com/zcash/halo2>.
- [95] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases. In *IEEE Symposium on Security and Privacy, SP*.
- [96] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for Outsourced Databases. In *CCS '15: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*.

Appendix

A	Additional Background	16
B	The Full Formalization of Idealized Protocols	19
C	Additional Details on Derived Operations	19
D	Additional Details on Our Compilation Results	21
E	Additional Details on How qedb Works	25
F	Queries Supported in qedb	30
G	Detailed results	30
H	Background on Multivariate Sumcheck and GKR	30
I	Typical Building Blocks in zkVMs	30
J	More on the Security of zkVMs	31
K	Contrasting Methodologies for VDBs: qedb vs General-Purpose SNARKs	31
L	Other Adopted Optimizations	32

A Additional Background

DEFINITION 2. A bilinear group is given by a description $\text{gk} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ with additive notation such that p is prime, so $\mathbb{F} = \mathbb{F}_p$ is a field. $\mathbb{G}_1, \mathbb{G}_2$ are cyclic (additive) groups of prime order p . We use the notation $[a]_1, [b]_2, [c]_t$ for elements in $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ respectively. $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a bilinear asymmetric map (pairing), which means that $\forall a, b \in \mathbb{Z}_p, e([a]_1, [b]_2) := [ab]_t$. We implicitly have that $[1]_t := e([1]_1, [1]_2)$ generates $\mathbb{G}_T$. We use $[a]_{1,2}$ to refer to two group elements $[a]_1 \in \mathbb{G}_1, [a]_2 \in \mathbb{G}_2$. In our constructions, we denote by $\mathcal{G}(p)$ the algorithm that, given as input the prime value p , outputs a description $\text{gk} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$.

The computational assumptions are:

DEFINITION 3. The q -DLOG assumption holds relative to $\mathcal{G}(1^\lambda)$ if for all PPT adversaries $\mathcal{A}$, the following is negligible:

$$\Pr \left[\tau \leftarrow \mathcal{A}(\text{gk}, \{[\tau^i]_1\}_{i=0}^q, \{[\tau^i]_2\}_{i=0}^q) \mid \text{gk} \leftarrow \mathcal{G}(1^\lambda); \tau \leftarrow \mathbb{F} \right].$$

DEFINITION 4 ([49]). The q -BSDH assumption holds relative to $\mathcal{G}(1^\lambda)$ if for all PPT adversaries $\mathcal{A}$, the following holds:

$$\Pr \left[\left(c, \frac{1}{\tau - c} e([1]_1, [1]_2) \right) \leftarrow \mathcal{A}(\text{gk}, \{[\tau^i]_1\}_{i=0}^q, \{[\tau^i]_2\}_{i=0}^q) \mid \text{gk} \leftarrow \mathcal{G}(1^\lambda); \tau \leftarrow \mathbb{F} \right] \leq \text{negl}(\lambda).$$

The AGM. The algebraic group model (AGM) [43] is an idealized model where the adversary is modeled as an *algebraic* algorithm. Algebraic algorithms may only compute group elements as linear combinations of group elements observed so far. Therefore, whenever they output a group element $X \in \mathbb{G}_1$, they also provide a representation $\{\alpha_i\}_{i=1}^N$ of $X = \sum_{i=1}^N [\alpha_i g_i]_1$ as a function of previously seen elements $[g_1], \dots, [g_N] \in \mathbb{G}_1$ of the same group.

Lagrange interpolation. Let $\mathbb{H} = \{x_1, \dots, x_n\}$ a multiplicative subgroup in a finite field $\mathbb{F}$, let $\phi(x) \in \mathbb{F}[x]$ a polynomial of degree at most $n - 1$, and let h_i be the evaluations $\phi(x_i), i = 1, \dots, n$.

Lagrange interpolation computes a linear combination of such evaluations to reconstruct the polynomial $\phi(x)$ defined as:

$$\phi(x) = \sum_{i=1}^n h_i \cdot \lambda_i(x)$$

where the *Lagrange interpolation polynomials* $\lambda_i(x)$ are given by

$$\lambda_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - h_j}{h_i - h_j}$$

A.1 Unique Setup

All the underlying building blocks we define rely on a single setup. We assume the following syntax for setups for ADS (and thus polynomial, vector commitments, and accumulators):

DEFINITION 5. A setup $\text{Setup}(1^\lambda, n) \rightarrow (\text{prk}, \text{vrk})$ is a probabilistic algorithm that takes as input the security parameter λ , a size bound n , and outputs a pair of keys (prk, vrk) (for proving and verification respectively). We assume λ implicitly provides a size parameter (e.g., vector size) for all our schemes.

A.2 Binding Commitments

All functional commitments have binding properties and a common syntax template for the commitment algorithm, which we now define. Below, a message space is “specialized” to the correct input size through a size parameter n .

DEFINITION 6 (COMMITMENT SCHEME). A commitment scheme $\text{Com} = (\text{Setup}, \text{AlgoCom})$ for a family of message spaces $\mathcal{M}$ is a pair of polynomial-time algorithms:

- Setup: as defined in Definition 5;
- $\text{AlgoCom}(\text{prk}, m) \rightarrow (\text{cm}, \text{aux})$: a deterministic algorithm that takes as inputs prk , an element $m \in \mathcal{M}_n$ and outputs the commitment cm and an auxiliary value aux .

A commitment scheme satisfies the following binding property.

Binding: For any security parameter $\lambda \in \mathbb{N}$, size parameter $n \in \mathbb{N}$ and PPT adversary $\mathcal{A}$, the following probability is negligible:

$$\Pr \left[\begin{array}{l} \text{cm}_{m_0} \leftarrow \text{AlgoCom}(1^\lambda, \text{prk}, m_0) \wedge \\ \text{cm}_{m_1} \leftarrow \text{AlgoCom}(1^\lambda, \text{prk}, m_1) \wedge \\ \text{cm}_{m_0} = \text{cm}_{m_1} \end{array} \mid \begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ (m_0, m_1) \leftarrow \mathcal{A}(1^\lambda) \\ m_0, m_1 \in \mathcal{M}_n \end{array} \right]$$

A.3 Polynomial commitments

A polynomial commitment scheme (PCS)

$$\text{PCom} = (\text{Setup}, \text{Commit}, \text{OpenPos}, \text{VfPos})$$

(with setup defined in Definition 5) consists of the following algorithms:

- $\text{Commit}(\text{prk}, \phi(x)) \rightarrow (\text{cm}, d)$ is a polynomial time algorithm that takes on input the public key prk and a polynomial $\phi(x)$ returns a commitment cm of $\phi(x)$ and a decommitment d .
- $\text{OpenPos}(\text{prk}, \phi(i), i, d) \rightarrow (i, \phi(i), \pi_i)$ is a polynomial time algorithm that on input $\text{prk}, \phi(i), i$ and the decommitment value d outputs a proof π_i for the evaluation of $\phi(x)$ in i , together with i and $\phi(i)$.
- $\text{VfPos}(\text{vrk}, \text{cm}, i, \phi(i), \pi_i) \rightarrow 0/1$ is a polynomial time algorithm (running in time sublinear in the size of ϕ) that takes in input the public key, the commitment, the evaluation $\phi(i)$, and the proof π_i and outputs 1 in case $\phi(i)$ is indeed the evaluation of the polynomial at index i and 0 otherwise.

A PCS is *succinct* if the size of commitments and evaluation proofs grows at most logarithmically with the degree of committed polynomials. A PCS satisfies (i) the binding property, meaning that it is infeasible for an adversary to find an opening cm to two different polynomials ϕ and ϕ' ; (ii) the evaluation binding, which captures the infeasibility for an adversary to prove two distinct evaluations of a committed polynomial on the same input; (iii) extractability, meaning that whenever an adversary outputs a valid commitment cm and a proof π for the evaluation y in x , there exists an efficient extractor that recovers ϕ such that $\phi(x) = y$ and cm is the commitment of ϕ ; (iv) homomorphism, that guarantees that given commitments cm_ϕ and cm_ψ of ϕ and ψ respectively, it is efficient to compute a new commitment $\text{cm}_{\phi+\psi}$ that is a valid commitment to the corresponding operation on the polynomials, without requiring access to the underlying coefficients.

Correctness For all λ , all size parameter n , for all $\phi \in \mathbb{F}[x]$ with $\deg(\phi) \leq n$, for any $i \in \mathbb{F}$, the following probability is negligibly close to 1.

$$\Pr \left[\begin{array}{l} \text{VfPos}(\text{vrk}, \text{cm}, i, \phi(i), w_i) = 1 \\ \text{cm}, d \leftarrow \text{Commit}(\text{prk}, \phi(x)) \\ (i, \phi(i), w_i) \leftarrow \text{OpenPos}(\text{prk}, \phi(i), i, d) \end{array} : (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \right].$$

Extractability: For all $\lambda \in \mathbb{N}$, for all PPT $\mathcal{A}$ there exists an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that this probability is at most negligible in λ .

$$\Pr \left[\begin{array}{l} \text{VfPos}(\text{vrk}, \text{cm}, i, \phi(i), w_i) = 1 \wedge (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ (\text{cm}, \cdot) \neq \text{Commit}(\text{prk}, \phi'(x)) \vee (i, \phi(i), w_i) \leftarrow \mathcal{A}(\text{prk}) \\ \phi'(i) \neq \phi(i) \end{array} : \phi'(x) \leftarrow \mathcal{E}_{\mathcal{A}}(\text{prk}, \text{cm}) \right].$$

A.4 Linear-map vector commitments

An LVC²³ scheme has these core capabilities: the ability to commit to a vector succinctly; the ability, given two commitments to vectors $\text{cm}_{\vec{u}}$ and $\text{cm}_{\vec{v}}$ to verify (and—on the prover side—prove) that $\langle \vec{u}, \vec{v} \rangle$ equals some claimed value y .

DEFINITION 7. An LVC scheme for vectors in $\mathbb{F}^n$ is a tuple of polynomial time algorithms $(\text{Setup}, \text{CommitVec}, \text{OpenLin}, \text{VerifyLin})$ (Setup is defined in Def. 5) that work as follows:

- $\text{CommitVec}(\text{prk}, \vec{v}) \rightarrow (\text{cm}, \text{aux})$ is a deterministic algorithm that on input a vector $\vec{v} = (v_1, v_2, \dots, v_m) \in M^m$ and the proving key prk , returns a commitment cm and auxiliary information aux .
- $\text{OpenLin}(\text{prk}, \text{aux}_{\vec{u}}, \text{aux}_{\vec{v}}, y) \rightarrow \pi_y$ is a deterministic algorithm that takes as input prk , auxiliary information about two (committed) vectors and outputs a proof π that $y = \langle \vec{u}, \vec{v} \rangle$.
- $\text{VerifyLin}(\text{vrk}, \text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi) \rightarrow 0/1$ is a deterministic algorithm (running in time sublinear in the size of the committed vectors) that takes as input the verification key vrk , commitments $\text{cm}_{\vec{u}}$ and $\text{cm}_{\vec{v}}$, a proof π and accepts or rejects.

²³Note to the reader: in our syntax we consider a restricted version of linear-map (inner product). We keep the terminology “linear map” because vector commitments that can prove inner products can be immediately lifted to ones proving general linear maps [22].

An LVC scheme satisfies the following properties: *Correctness* requires that if a prover commits to two vectors and computes their inner product honestly, then the proof they generate will always convince the verifier of the correct result; *binding* property means that it is infeasible for an adversarial committer to produce a commitment cm and two valid openings $\vec{u}$ and $\vec{v}$; *Homomorphic* property means that given commitments $\text{cm}_{\vec{u}}$ to $\vec{u}$ and $\text{cm}_{\vec{v}}$ to $\vec{v}$, it must be efficient to compute a commitment $\text{cm}_{\vec{u}+\vec{v}}$ by computing $\text{cm}_{\vec{u}+\vec{v}} = a \cdot \text{cm}_{\vec{u}} + b \cdot \text{cm}_{\vec{v}}$; finally, *extractability* property means that there exists an efficient extractor $\mathcal{E}$ such that, given a commitment $\text{cm}_{\vec{u}}$ and oracle access to an adversarial prover P^* that produces a proof π accepted by the verification algorithm VerifyLin for some commitment $\text{cm}_{\vec{v}}$ and scalar y , the extractor can recover a vector $\vec{u}'$ such that $\text{CommitVec}(\vec{u}') = \text{cm}_{\vec{u}} \wedge \langle \vec{u}', \vec{v} \rangle = y$ with all but negligible probability. More formally:

LVC Correctness: An LVC scheme is perfectly correct if for all $\lambda \in \mathbb{N}$, all n , and any $\vec{u}, \vec{v} \in \mathbb{F}^n$,

$$\Pr \left[\begin{array}{l} (\text{vrk}, \text{prk}) \leftarrow \text{Setup}(1^\lambda, n) \\ \text{VerifyLin}(\text{vrk}, \text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi_y) = 1 \\ \text{cm}_{\vec{v}}, y, \pi_y \leftarrow \text{CommitVec}(\text{prk}, \vec{v}) \end{array} : \begin{array}{l} (\text{cm}_{\vec{u}}, \text{aux}_{\vec{u}}) \leftarrow \text{Commit}(\text{prk}, \vec{u}) \\ (\text{cm}_{\vec{v}}, \text{aux}_{\vec{v}}) \leftarrow \text{Commit}(\text{prk}, \vec{v}) \\ \pi_y \leftarrow \text{OpenLin}(\text{prk}, \text{aux}_{\vec{u}}, \text{aux}_{\vec{v}}, y) \end{array} \right] = 1$$

Extractability: For all $\lambda \in \mathbb{N}$, for all PPT $\mathcal{A}$ there exists an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that the following is negligible:

$$\Pr \left[\begin{array}{l} \text{VerifyLin}(\text{vrk}, \text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi) = 1 \\ \wedge \langle \vec{u}^*, \vec{v}^* \rangle \neq y \\ \vee (\text{cm}_{\vec{u}}, \cdot) \neq \text{Commit}(\text{prk}, \vec{u}^*) \\ \vee (\text{cm}_{\vec{v}}, \cdot) \neq \text{Commit}(\text{prk}, \vec{v}^*) \end{array} : \begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ (\text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi) \leftarrow \mathcal{A}(\text{prk}) \\ (\vec{u}^*, \vec{v}^*) \leftarrow \mathcal{E}_{\mathcal{A}}(\text{prk}, \text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}) \end{array} \right]$$

Homomorphism of Commitments: Given two vectors $\vec{u}$ and $\vec{v}$ and any scalars a, b it holds that if $\vec{w} = a \cdot \vec{u} + b \cdot \vec{v}$ then $\text{Commit}(\text{prk}, \vec{w}) = a \cdot \text{cm}_{\vec{u}} + b \cdot \text{cm}_{\vec{v}}$, where $\text{cm}_{\vec{u}}$ and $\text{cm}_{\vec{v}}$ are respectively the commitments of $\vec{u}$ and $\vec{v}$.

We now prove our variant of the LVC scheme from [22]. In contrast to the original construction we let the verifier take as input two vectors as committed inputs. In contrast, the original construction lets the verifier take as input the linear function as a vector in the clear. This would have caused a loss of succinctness in some cases of our scheme. The change is relatively minor and requires the prover to send one more group element and the verifier to perform one more pairing check.

PROOF OF THEOREM 4.2. For the LVC construction in Section 4:

- Completeness follows by inspection.
- The homomorphic property follows from the following observation. Given the commitments $\text{cm}_{\vec{u}} = \sum_{i=1}^N u_i[\lambda_i(\tau)]_1$ $\text{cm}_{\vec{v}} = \sum_{i=1}^N v_i[\lambda_i(\tau)]_1$ respectively of $\vec{u}$ and $\vec{v}$, let $\vec{w} = a \cdot \vec{u} + b \cdot \vec{v}$, the commitment $\text{cm}_{\vec{w}}$ of $\vec{w}$ is $a \cdot \text{cm}_{\vec{u}} + b \cdot \text{cm}_{\vec{v}}$.
- The scheme satisfies binding under q-DLOG assumption in the AGM (Definition 4).
- The scheme satisfies extractability in the AGM model under the q-BSDH assumption. We follow and expand the proof in [22, Thm. 6]. The proof proceeds through a sequence of games. The first game G_0 is the original game of the binding game (Def. A.2). $\mathcal{A}$ outputs $(\text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi)$ and

$(\text{cm}_u, \text{cm}_{v'}, y', \pi')$ with $y \neq y'$ but both verify it is possible to extract $\vec{v}$ and $\vec{v}'$ from cm_v and $\text{cm}_{v'}$. G_1 is the same as G_0 but aborts if $\deg(R) > m-2$ where $R(x)$ is the algebraic representation of $[R]_1$ from π . If G_0 and G_1 are distinguishable we can construct an adversary $\mathcal{B}$ that solves qSDH by extracting high-degree terms from $R(x)$. The advantage of $\mathcal{A}$ in G_1 is negligible, indeed, given $\text{cm}_u(x) = \sum_{j=1}^m u_j \lambda_j(x) + x^m \hat{u}$ and $P(x) = \text{cm}_u(x) \text{cm}_v(x) - m^{-1}y - xR(x) - t(x)Q(x)$, the verification equation implies $P(\tau) = 0$. Therefore there are two cases:

Case 1: $P(x) = 0$. Then:

$$\begin{aligned} & \sum_{j=1}^m u_j v_j \lambda_j(x) + x^m \hat{u} \sum_{j=1}^m v_j \lambda_j(x) = m^{-1}y + xR(x) \\ \implies & \sum_{j=1}^m u_j v_j \lambda_j(0) = m^{-1}y \quad (\text{evaluate at } 0) \\ \implies & \sum_{j=1}^m u_j v_j = y \quad (\text{since } \lambda_j(0) = m^{-1}) \end{aligned}$$

Thus there exists $\vec{u}$ with $\langle \vec{u}, \vec{v} \rangle = y$ and $\mathcal{A}$ loses.

Case 2: $P(x) \neq 0$ but $P(\tau) = 0$. We build $\mathcal{B}$ against DLOG:

- (1) Given $[\tau]_1$, $\mathcal{B}$ computes roots of $P(x)$;
 - (2) Checks which root τ' of $P(x)$ satisfies $[\tau']_1 = [\tau]_1$.
- The only point that remains to be proven is that the value $\widetilde{\text{cm}}_v$ in π and the value $\widetilde{\text{cm}}_{v'}$ in π' are equal to the values cm_v and $\text{cm}_{v'}$ received in input from VFLin, but it follows from the last check $e(\text{cm}_v, [1]_2) = e([1]_1, \widetilde{\text{cm}}_v)$ of VFLin.

□

A.5 Subvector-opening vector commitments

DEFINITION 8. A VC scheme for vectors in $\mathbb{F}^n$ is a tuple of polynomial time algorithms (Setup, CommitVec, OpenSub, VerifySub) (with Setup defined in Def.5) that work as follows:

- $\text{OpenSub}(\text{prk}, \text{aux}_{\vec{u}}, I) \rightarrow \pi$ is a deterministic algorithm that takes as input prk , a vector $\vec{u}$, a set of indices I and outputs a proof π .
- $\text{VerifySub}(\text{vrk}, \text{cm}_u, \vec{v}, I, \pi) \rightarrow 0/1$ is a deterministic algorithm (running in time sublinear in the size of the committed vector $\vec{u}$) that takes as input the verification key vrk , commitments cm_u , a subvector $\vec{v}$, a set of indices I , a proof π and accepts or rejects.

CommitVec is the same used by LVC (see Def.7). VC scheme satisfies all the properties of LVC (see Def.7) where only correctness and extractability change as follows:

The properties of a Vector Commitment (VC) scheme match those of an LVC scheme, except that: (i) correctness requires that if the prover commits to a vector $\vec{v}$ and opens a subvector $\vec{u}$ at indices $I \subseteq [\bar{n}]$, then the proof from OpenSub always convinces the verifier that $\vec{u}$ is correct; (ii) extractability requires an efficient extractor $\mathcal{E}$ such that, given a commitment cm_u and oracle access to an adversary $\mathcal{P}^*$ outputting a proof π accepted by VerifySub for some I , it can recover $\vec{u}'$ satisfying $\text{CommitVec}(\text{prk}, \vec{u}') = \text{cm}_u$ with all but negligible probability.

Correctness For all $\lambda \in \mathbb{N}$, $\vec{v} \in \mathbb{F}^n$, $X := \{j_1, \dots, j_\ell\} \subseteq [n]$, $\vec{w} \in \mathbb{F}^\ell$ such that $(v_{j_1}, \dots, v_{j_\ell}) = (w_1, \dots, w_\ell)$, the following probability is negligibly close to 1:

$$\Pr \left[\begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ \text{VerifySub}(\text{vrk}, \text{cm}_{\vec{v}}, \vec{w}, X, \pi) = 1 : (\text{cm}_{\vec{v}}, \text{aux}_{\vec{v}}) \leftarrow \text{CommitVec}(\text{prk}, \vec{v}) \\ \pi \leftarrow \text{OpenSub}(\text{prk}, \text{aux}_{\vec{v}}, X) \end{array} \right].$$

Extractability: For all $\lambda \in \mathbb{N}$, for all PPT $\mathcal{A}$ there exists an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that the following is negligible:

$$\Pr \left[\begin{array}{l} \text{VerifySub}(\text{vrk}, \text{cm}_{\vec{v}}, \vec{w}, X, \pi) = 1 \wedge (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ (\text{cm}_{\vec{v}} \neq \text{CommitVec}(\text{prk}, \vec{v}) \vee (\text{cm}_{\vec{v}}, X := \{j_1, \dots, j_\ell\}, \pi) \leftarrow \mathcal{A}(\text{prk}) \\ (v_{j_1}, \dots, v_{j_\ell}) \neq (w_1, \dots, w_\ell) \quad \vec{v} \leftarrow \mathcal{E}_{\mathcal{A}}(\text{prk}, \text{cm}_{\vec{v}}) \end{array} \right].$$

A.6 Set accumulators

DEFINITION 9. A Set Accumulator ²⁴ (SA) is a tuple of PPT algorithms (Setup, Accum, OpenOp, VfOp) (with Setup algorithm defined in Definition 5) such that:

- $\text{Accum}(\text{prk}, S) \rightarrow \text{acc}$ is a deterministic algorithm that takes as inputs a set S and prk and outputs the accumulator value acc ;
- $\text{OpenOp}(\text{prk}, S_1, S_2, \text{op}) \rightarrow (S = \text{op}(S_1, S_2), \pi_{\text{op}})$ is a deterministic algorithm that takes as inputs sets S_1, S_2 , an operation $\text{op} \in \{\subseteq, \cap, \cup, \setminus\}$, and outputs a proof π_{op} that S obtained by applying op to S_1 and S_2 .
- $\text{VfOp}(\text{vrk}, \text{acc}_S, \text{acc}_{S_1}, \text{acc}_{S_2}, \text{op}, \pi_{\text{op}}) \rightarrow 0/1$ is a deterministic algorithm (running in time sublinear in the size of the accumulated sets) that takes as inputs an accumulator acc_S , two accumulators $\text{acc}_{S_1}, \text{acc}_{S_2}$, an operation $\text{op} \in \{\subseteq, \cap, \cup, \setminus\}$, a proof π_{op} that the set in acc_S is given applying op to acc_{S_1} and acc_{S_2} , and accepts or rejects.

We require that a set accumulator scheme (Accum, OpenOp, VfOp) is correct, which means that for correctly computed opening operation, the relative verify operation accepts. Moreover, for an adversary it is computationally infeasible to find $W \not\subseteq S$ (resp. $W \neq S \cap T$, $W \neq S \cup T$, $W \neq S \setminus T$) and a proof π_W such that the associated verification procedure accepts. The set accumulator must achieve extractability, meaning that given accepting proofs from an adversarial prover, the probability to extract values from accumulators that are incorrect is negligible. We also require that a set accumulator is homomorphic, i.e., guarantees that given accumulators acc_{S_1} and acc_{S_2} of sets S_1 and S_2 respectively, it is efficient to compute a new accumulator $\text{acc}_{S_1 \cup S_2}$ that is a valid accumulator for the corresponding sets.

Correctness For all λ , all n, S_1, S_2 sets of elements in $\mathbb{F}$ of size at most n , the following probability is overwhelming:

$$\Pr \left[\begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ \text{VfOp}(\text{vrk}, \text{acc}_S, \text{acc}_{S_1}, \text{acc}_{S_2}, \text{op}, \pi_{\text{op}}) = 1 : \begin{array}{l} \text{acc}_{S_1} \leftarrow \text{Accum}(\text{prk}, S_1) \\ \text{acc}_{S_2} \leftarrow \text{Accum}(\text{prk}, S_2) \\ (\text{acc}_S, \pi_{\text{op}}) \leftarrow \text{OpenOp}(\text{prk}, S_1, S_2, \text{op}) \\ S = \text{op}(S_1, S_2) \end{array} \end{array} \right].$$

²⁴Note to the reader: we do not require membership proofs from accumulator schemes, only set comparison operations.

Extractability: For all λ , all n , for all PPT $\mathcal{A}$ there exists an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that the following probability is negligible:

$$\Pr \left[\begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda, n) \\ (\text{acc}_S, \text{acc}_{S_1}, \text{acc}_{S_2}, \text{op}, \pi_{\text{op}}) \leftarrow \mathcal{A}(\text{prk}) \\ (S, S_1, S_2) \leftarrow \mathcal{E}_{\mathcal{A}}(\text{prk}, \text{acc}_S, \text{acc}_{S_1}, \text{acc}_{S_2}) \end{array} \right]$$

where the event $E_{\text{acc,win}}^{\mathcal{A}}$ is defined as:

$$E_{\text{acc,win}}^{\mathcal{A}} := \begin{cases} \text{VfOp}(\text{vrk}, \text{acc}_S, \text{acc}_{S_1}, \text{acc}_{S_2}, \text{op}, \pi_{\text{op}}) = 1 \wedge \\ \text{acc}_S \neq \text{Accum}(\text{prk}, \text{op}(S_1, S_2)) \vee \\ \text{acc}_{S_1} \neq \text{Accum}(\text{prk}, S_1) \vee \\ \text{acc}_{S_2} \neq \text{Accum}(\text{prk}, S_2) \end{cases}$$

B The Full Formalization of Idealized Protocols

We now formalize the semantics of idealized VDBs. While we focus on a non-interactive setting where the idealized prover sends a single batch of handles, the framework naturally extends to multiple rounds. Moreover, if the components used to compile an idealized VDB are non-interactive, then the resulting cryptographic VDB is non-interactive as well. Recall that $Q(\cdot, \cdot)$ denotes the set of valid responses to a query (see also Fig. 6).

DEFINITION 10 (IDEALIZED PROVER/VERIFIER AND THEIR INTERACTION). We denote by an idealized algorithm (prover or verifier) a randomized²⁵ machine with access to special oracles, described in Fig. 12. We denote the verifier’s decision at the end of the interaction flow (composed of local oracle invocations and a single round of message passing) in Fig. 12 as

$$b \leftarrow \langle \text{p}^{\text{Op}}, \text{vf}^{\text{Op}} \rangle(q, \text{resp})$$

where $b \in \{0, 1\}$ denotes rejection or acceptance.

REMARK 3. We assume all algorithms implicitly take as input a security parameter λ . For this idealized model this will play uniquely the role of a statistical security parameter (see Footnote 25). We will also implicitly assume it to be useful as offering some size bound parameter (polynomial in λ for some fixed polynomial) to the honest algorithms (e.g., maximum size of data structures to allocate internally, etc).

Completeness For all $\text{db} \in \mathcal{DB}$, for all queries q and responses resp s.t. $\text{resp} \in Q(q, \text{db})$, then the following probability is overwhelming:

$$\Pr \left[\langle \text{p}^{\text{Op}}, \text{vf}^{\text{Op}} \rangle(q, \text{resp}) = 1 : \begin{array}{l} (\mathbb{T}_{\parallel}^{\text{handles}}, \mathbb{T}_{\circlearrowleft}^{\text{handles}}) \leftarrow \text{Idx}(\text{db}) \\ \text{Op}, \text{Opf} \text{ as in Fig. 12} \end{array} \right]$$

Soundness For all idealized provers $\tilde{\text{P}}$, for all $\text{db} \in \mathcal{DB}$, for all queries q and responses resp s.t. $\text{resp} \notin Q(q, \text{db})$, then the following

²⁵The honest idealized prover/verifier should operate in polynomial time. However, our notion of soundness will be quite strong and state that the protocol is secure even against non polynomial time machines. This is similar to what is achieved in other idealized models such as [2, 8, 12, 19, 31, 45].

probability is negligible:

$$\Pr \left[\langle \tilde{\text{p}}^{\text{Op}}, \text{vf}^{\text{Op}} \rangle(q, \text{resp}) = 1 : \begin{array}{l} (\mathbb{T}_{\parallel}^{\text{handles}}, \mathbb{T}_{\circlearrowleft}^{\text{handles}}) \leftarrow \text{Idx}(\text{db}) \\ \text{Op}, \text{Opf} \text{ as in Fig. 12} \end{array} \right]$$

The notion of succinctness of an idealized VDB captures the fact that the “non-oracle” communication complexity of the protocol does not “go beyond” the strictly necessary (the size of the query and the response). We constrain the verification time analogously.

DEFINITION 11 (SUCCINCTNESS OF AN IDEALIZED PROTOCOL). Consider the interaction with the honest prover in Fig. 12 and let $n_{\text{comm}} = |\pi_{\text{hint}}| + n_{\text{resp}}$ be the total communication complexity of the protocol (with n_{resp} being the total size of elements read through the interaction by the verifier). We say that an idealized protocol is succinct if for any database and query $n_{\text{comm}} = \tilde{O}(|\text{resp}| + |\text{qry}|)$ where resp is a correct response to the query. The verification time is also required to be bounded by $\tilde{O}(|\text{resp}| + |\text{qry}|)$ for any database and query.

C Additional Details on Derived Operations

We present the full list of derived operations from the core set operations shown in Section 6.1, where the nullifying test has already been described.

REMARK 4 (A NOTE ON NUMERICAL TYPES AND RANGE CHECKS). We assume that sets and slices are defined over a field. Nonetheless when performing range queries or other additions we will be working on bounded integers. We will make two assumptions (which are easy to instantiate and have also been used in prior work on VDBs) that will make our approaches based on homomorphism work: the fact that at preprocessing time commitments to integer values are generated honestly and will be limited to a certain range; the fact that the number of homomorphic operations required by any given query will not go beyond that range even in the worst case. Finally, we remind the reader that given a test of the type $v \stackrel{?}{\geq} 0^{27}$, it is easy, through homomorphism, to obtain tests for $v \stackrel{?}{\geq} \alpha$ (which becomes $v - \alpha \cdot 1 \stackrel{?}{\geq} 0$) and for $v \leq \alpha$ (which becomes $-v \stackrel{?}{\geq} -\alpha$).

In producing our derived operations we use the following core techniques:

- *homomorphism*: this is often used to check whether two vectors are equal in a set of positions by subtracting them; we also apply homomorphism to produce a vector of values all equal to the same value α by, e.g., multiplying α by the slice handle of all ones.
- *complement set and “partition trick”*: when checking that a property is true in a target set X and only in that set, we let the Prover send (a handle to) the set complement set $\bar{X}$ and check the property is true in X but false in $\bar{X}$.
- *incrementality*: we occasionally use each of the new derived tests as a building blocks in others.

A list of the derived operations and arguments for their correctness.

Nullifying test The key insight is that by checking the difference vector Δ is zero on the complement of the target subset X_0 , combined with checking that $\bar{u}$ is zero on X_0 , the verifier ensures the property without needing to explicitly compute

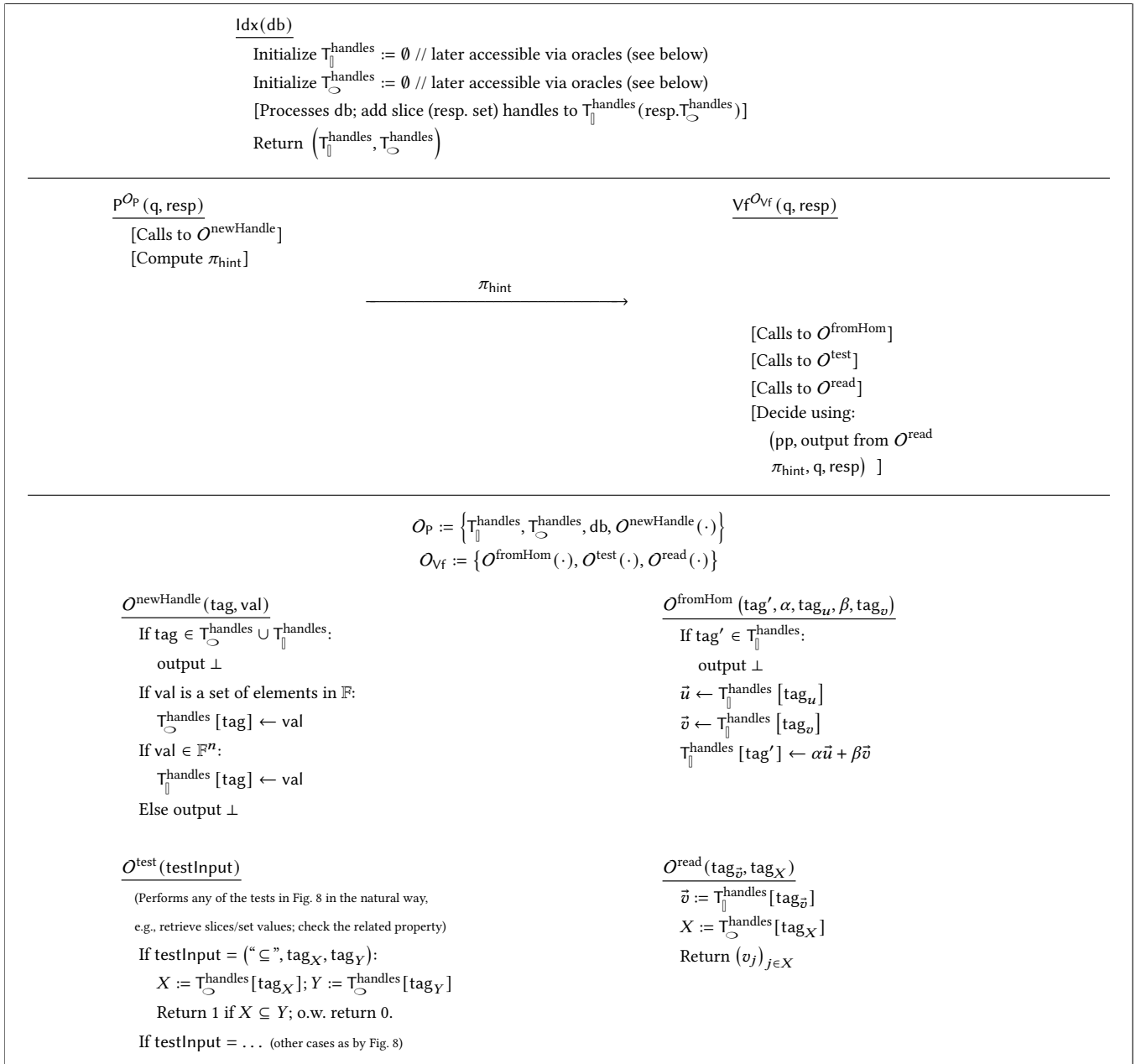


Figure 12: Formal details for the idealized protocol interaction. We assume that any of the oracles returns $\perp$ whenever trying to access a non-existing tag in the tables. Notice that, by placing them among its oracles, the prover has RAM access to db and the preindexed tables $T_{\parallel}^{\text{handles}}$ and $T_{\circ}^{\text{handles}}$. The implementation of O^{test} is straightforward and above we just provide an example that is immediate to extend.

the indicator function. Notice that the checks correspond exactly to the property on the left and correctness and soundness hold unconditionally.

Range check Intuitively, this test works by letting the prover send slices of bits that, when appropriately scaled (by respective powers of two) and summed, can be seen to provide the

target slice $\vec{v}$. The verifier can check the “bit-ness” of the received slices by having the prover send the handle sets of positions where these slices are equal to 0; the verifier then applies zeroing (the instruction we described in the previous bullet) in these claimed positions and checks equality to the claimed bit slices.

Strict sign check Here we apply this observation: the subvector $\vec{v}_{X_>}$ is positive everywhere if and only if $\vec{v}_{X_>} - \vec{1}$ is non-negative everywhere. We emulate the “subvectoring” operation by nullifying anything outside $X_>$. The verifier then applies the range check from the previous item.

Positions where two slices are equal First observe that two slices are equal in the set of positions X_0 if and only there if and only *i*) if their difference is zero in X_0 and *ii*) non-zero everywhere else. Testing *i*) is straightforward (zero-testing is a core operation). To check *ii*) we first let the prover “declare” what positions are non-zero and positive and which positions are non-zero and negative. We then apply the test from the last item twice (to check that a set of positions is strictly negative we first negate the slice through homomorphism and then check it is strictly positive).

Sumcheck within a target subset X We let the prover send the indicator vector of X (which is appropriately checked by the verifier) and then we apply an inner product between slices (a core operation).

Pre-image check Here, given a value α we want to test whether it is true that the position in which a slice $\vec{v}$ assumes value α corresponds exactly to a claimed set of positions X_α (we call it a *preimage* check because this is the preimage of the value α when seeing the slice as a function $\vec{v} : [n] \rightarrow \mathbb{F}$). This check will be useful in JOINS and is a natural tool in other SQL queries. Its implementation is straightforward given one the derived check for positions where two slices are equal: the verifier produces a slice $(\alpha, \alpha, \dots, \alpha)$ and then checks that the latter and $\vec{v}$ assume equal values in all and only the positions in X_α .

The full list of derived operations in the information-theoretic model is shown in Table 2.

D Additional Details on Our Compilation Results

Here we provide an expanded and more detailed discussion of compilation. Some of the content in this section will overlap with that presented in Section 5.2.

Our idealized interface in Fig. 8 resembles (to an extent) the functionalities offered by authenticated data structures like *vector commitments* [25] and *set accumulators* [72]. Vector commitments allow one to commit to a vector of values and later open a subset J of positions via succinct proofs—a property known as *subvector opening* if the proofs are of length sublinear in $|J|$ —while set accumulator allows one to compute a digest of a set of values and to succinctly prove set membership; occasionally (as well as in this work) accumulators can prove relations about accumulated sets succinctly, such as subset or intersection.

Standard accumulators and vectors commitments *almost* match the operations in Fig. 8. As we will soon explain, we can extend the functionalities of these cryptographic primitives (hereby referred for brevity as ADS) to obtain an *exact* match with the operations required in idealized scheme. Let us first observe that, given the right primitives, there is an intuitive compilation strategy for transforming an idealized VDB into a cryptographic one.

The structure of our compiler is as follows. Starting from an idealized protocol that captures verification for a set of queries (i.e., is complete and sound for them), the cryptographic prover, for each set handle (resp. slice handle) sent by the idealized prover, sends the corresponding set accumulator (resp. vector commitment) during VDB execution. Whenever the idealized verifier performs the abstract operations, the prover supplies the corresponding proof using the related ADS interface. Relations among sets (e.g., subset) follow directly from the interface of set accumulators and homomorphism is immediate under homomorphic vector commitments. We can perform a **read** via subvector openings (with a minor additional tweak).

The remaining operations (inner product and zero test) require new definitions and, in part, changes in existing constructions. The primitive closest to the inner product check is the Linear-map Vector Commitments (LVC) of [22]. However, it is not a precise match as their interface does not take a commitment to a second vector, assuming the verifier knows it in full. To the best of our knowledge, zero-testing proofs (which, given an accumulator to X and a vector commitment to $\vec{v}$, guarantee $v_j = 0$ for every $j \in X$) have no explicit analogue in any prior literature.

Our work provides the first formal definitions for these properties. In particular, zero-testing (which takes an accumulator and a commitment to a vector as input) is, to our knowledge, the first example of formal requirement linking vector commitments and accumulators. Since it is a natural notion, it may plausibly be of independent interest and useful in other settings. We stress that, in our context, these properties are key to obtain the succinctness of the final cryptographic VDB (for example, in the case of zero-testing, because otherwise the prover would need to send full openings to the sets in order to check for zeros). For the same reasons, in all these notions, we require the verifier to run in time sublinear in the size of the committed objects.

Given the discussion above, the final minimal requirements for our compiler are two primitives Π_{LVC} and Π_{Acc} , respectively a homomorphic LVC scheme (supporting our flavor of inner product test) and a set accumulator supporting set relations, and such that *together* they support zero-testing proofs. We show how to instantiate them in two ways—a more concrete and efficient instantiation and a more generic one (see also Fig. 1):

- *From pairings*: we show how the pairing-based LVC from [22] and set accumulators from [76] can, with a few modifications, match our interface. These two primitives are good candidates not only because of their efficient profile, but also from a technical standpoint in order to obtain our special properties: both primitives can be seen as committing (under the hood) to a vector/set through a KZG-style commitment [57], a popular polynomial commitment scheme.
- *From any extractable homomorphic polynomial commitment*: we observe that the patterns that allow one to obtain our ADSs through KZG-style primitives are actually more general. Thus, at the price of a slightly less efficient instantiation, we can show that a homomorphic polynomial commitment suffices to build our special primitives²⁶. This result

²⁶The resulting scheme is slightly less efficient than the direct pairing-based instantiation since pairings offer shortcuts not available with an abstract polynomial commitment interface. In the latter case, we must occasionally provide extra evaluation proofs

"Nullifying" test $\llbracket \vec{u} \rrbracket \stackrel{?}{=} \llbracket \vec{v} \rrbracket \left[\mathcal{X}_0 \rightarrow 0 \right]$ i.e., $\forall j \ u_j \stackrel{?}{=} v_j \cdot (1 - \mathbb{1}_{X_0}(j))$	Prover sends: $\mathcal{X}_0 := \mathcal{S}^* \setminus \mathcal{X}_0$ Verifier defines $\llbracket \Delta \rrbracket := \llbracket \vec{u} \rrbracket - \llbracket \vec{v} \rrbracket$ and checks: • $\mathcal{X}_0 \stackrel{?}{=} \mathcal{S}^* \setminus \mathcal{X}_0$ • $\llbracket \vec{u} \rrbracket [\mathcal{X}_0] \stackrel{?}{=} 0$ (zero test in X_0) • $\llbracket \Delta \rrbracket [\mathcal{X}_0] \stackrel{?}{=} 0$ (equality outside X_0)
Range check $\llbracket \vec{v} \rrbracket \in [0, 2^\ell)$	Define $v_j^{(i)}$ as i-th bit of v_j, so $v_j = \sum_i 2^{i-1} v_j^{(i)}$. Let $\vec{v}^{(i)} := (v_1^{(i)}, \dots, v_m^{(i)})$, $m = \vec{v}$. Define $X_0^{(i)} := \{j : v_j^{(i)} = 0\}$. Prover sends $\llbracket \vec{v}^{(1)} \rrbracket, \dots, \llbracket \vec{v}^{(\ell)} \rrbracket$ and $\mathcal{X}_0^{(1)}, \dots, \mathcal{X}_0^{(\ell)}$. Verifier checks: $\llbracket \Delta \rrbracket := \sum_i 2^{i-1} \llbracket \vec{v}^{(i)} \rrbracket - \llbracket \vec{v} \rrbracket, \quad \llbracket \Delta \rrbracket [\mathcal{S}^*] \stackrel{?}{=} 0$ and that each $\llbracket \vec{v}^{(i)} \rrbracket \stackrel{?}{=} \llbracket \vec{1} \rrbracket [\mathcal{X}_0^{(i)} \rightarrow 0]$.
Strict sign check within target subset $\llbracket \vec{v} \rrbracket [\mathcal{X}_>] \stackrel{?}{>} 0$	Define indicator vector $\vec{u}_1$ for $X_>$, and vector $\vec{u}_{\text{zero}}$ such that $u_{\text{zero},j} = v_j$ if $j \in X_>$ else 0. Prover sends $\llbracket \vec{u}_1 \rrbracket, \llbracket \vec{u}_{\text{zero}} \rrbracket, \mathcal{X}_> := \mathcal{S}^* \setminus \mathcal{X}_>$. Verifier checks: • $\llbracket \vec{u}_1 \rrbracket \stackrel{?}{=} \llbracket \vec{1} \rrbracket [\mathcal{X}_> \rightarrow 0]$ • $\llbracket \vec{u}_{\text{zero}} \rrbracket \stackrel{?}{=} \llbracket \vec{v} \rrbracket [\mathcal{X}_> \rightarrow 0]$ • $\llbracket \vec{v}_{\geq 0} \rrbracket := \llbracket \vec{u}_{\text{zero}} \rrbracket - \llbracket \vec{u}_1 \rrbracket \stackrel{?}{\geq} 0$ • $\mathcal{X}_> \stackrel{?}{=} \mathcal{S}^* \setminus \mathcal{X}_>$
Tests where two slices are equal $\mathcal{X}_0 \stackrel{?}{=} \text{eqSet}(\llbracket \vec{u} \rrbracket, \llbracket \vec{v} \rrbracket)$ i.e., we test: $X_0 \stackrel{?}{=} \{j : u_j = v_j\}$	Define sets $X_+ := \{j : u_j > v_j\}$ and $X_- := \{j : u_j < v_j\}$. Prover sends $\mathcal{X}_+, \mathcal{X}_-$. Verifier defines $\llbracket \Delta \rrbracket := \llbracket \vec{u} \rrbracket - \llbracket \vec{v} \rrbracket$ and checks: • $\mathcal{X}_0, \mathcal{X}_+, \mathcal{X}_-$ partition $\mathcal{S}^*$ • $\llbracket \Delta \rrbracket [\mathcal{X}_0] \stackrel{?}{=} 0$ • $\llbracket \Delta \rrbracket [\mathcal{X}_+] \stackrel{?}{>} 0$ • $-\llbracket \Delta \rrbracket [\mathcal{X}_-] \stackrel{?}{>} 0$
Sum check within target subset $\sum_{j \in \mathcal{X}} \llbracket v_j \rrbracket \stackrel{?}{=} y$	Define indicator vector $\vec{u}_1$ for X. Prover sends $\llbracket \vec{u}_1 \rrbracket$ and $\mathcal{X} := \mathcal{S}^* \setminus \mathcal{X}$. Verifier checks: • $\llbracket \vec{u}_1 \rrbracket \stackrel{?}{=} \llbracket \vec{1} \rrbracket [\mathcal{X} \rightarrow 0]$ • $\langle \llbracket \vec{v} \rrbracket, \llbracket \vec{u}_1 \rrbracket \rangle \stackrel{?}{=} y$ • $\mathcal{X} \stackrel{?}{=} \mathcal{S}^* \setminus \mathcal{X}$
Pre-image check $\mathcal{X}_\alpha \stackrel{?}{=} \alpha^{-1}(\llbracket \vec{v} \rrbracket)$	Define $\mathcal{X}_\alpha := \{j : v_j = \alpha\}$ where $\alpha \in \mathbb{F}$. Verifier defines $\llbracket \vec{u}_\alpha \rrbracket := \alpha \llbracket \vec{1} \rrbracket$ and checks: $\mathcal{X}_\alpha \stackrel{?}{=} \text{eqSet}(\llbracket \vec{u}_\alpha \rrbracket, \llbracket \vec{v} \rrbracket)$

Table 2: How to obtain derived operations in our model (see also Fig. 8). Both the universe of rows $*$ and the slice corresponding to the constant vector of all ones (denoted by $\vec{1}$) are assumed to be part of the indexing.

allows us to obtain instantiations from other assumptions (for example, post-quantum secure ones through lattices; see also Section 5.2).

D.1 Accumulators and LVC with Zero-Testing: Definition

We assume a vector and set commitment schemes (resp. Def 9 and Def. 9) sharing the same Setup and both binding in their respective domain. The vector size $\bar{n}$ is assumed to be polynomial in λ .

The property we are interested in can be thought of as an “extended” primitive of both accumulators and (subvector-opening) vector commitments: intuitively, this primitive is such that given $\text{cm}_{\vec{v}}$ and acc_X where $X \subseteq [\bar{n}]$, then a prover is able to convince a verifier that $v_j = 0$ for all $j \in X$.

DEFINITION 12. *Formally, we say that an LVC (Appendix A.4) and SA (Appendix A.6) scheme (jointly) support zero-testing on accumulated sets if there exist the following two efficient algorithms satisfying the properties below (the $\star$ is just to stress that the algorithms are new and not standard in this type of authenticated data structures):*

- $\text{PrSubvec0}^\star(\text{prk}, \text{aux}_{\vec{v}}, \text{aux}_X) \rightarrow \pi$
- $\text{VfSubvec0}^\star(\text{vrk}, \text{cm}_{\vec{v}}, \text{acc}_X, \pi) \rightarrow 0/1$

Correctness For all $\lambda \in \mathbb{N}$, $\vec{v} \in \mathbb{F}^{\bar{n}}$, $X \subseteq [\bar{n}]$ such that $\vec{v}_j = 0$ for all $j \in X$, it holds that the following probability is negligibly close to 1:

$$\Pr \left[\begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda) \\ \text{VfSubvec0}^\star(\text{vrk}, \text{cm}_{\vec{v}}, \text{acc}_X, \pi) = 1 : \begin{array}{l} (\text{cm}_{\vec{v}}, \text{aux}_{\vec{v}}) \leftarrow \text{VC.Com}(\text{prk}, \vec{v}) \\ (\text{acc}_X, \text{aux}_X) \leftarrow \text{Acc.Com}(\text{prk}, X) \\ \pi \leftarrow \text{PrSubvec0}^\star(\text{prk}, \text{aux}_{\vec{v}}, \text{aux}_X) \end{array} \end{array} \right]$$

Extractability For all $\lambda \in \mathbb{N}$, for all PPT $\mathcal{A}$ there exists an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that this probability is at most negligible in λ :

$$\Pr \left[\begin{array}{l} \text{VfSubvec0}^\star(\text{vrk}, \text{cm}_{\vec{v}}, \text{acc}_X, \pi) = 1 \\ \wedge \left(\begin{array}{l} \text{acc}_X \neq \text{Acc.Com}(\text{prk}, X) \\ \vee \text{cm}_{\vec{v}} \neq \text{VC.Com}(\text{prk}, \vec{v}) \\ \vee \exists j \in X : v_j \neq 0 \end{array} \right) : \begin{array}{l} (\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda) \\ (\text{cm}_{\vec{v}}, \text{acc}_X, \pi) \leftarrow \mathcal{A}(\text{prk}) \\ (\vec{v}, X) \leftarrow \mathcal{E}_{\mathcal{A}}(\text{prk}) \end{array} \end{array} \right]$$

D.2 Instantiating the zero-testing property with polynomial commitments

We now present a general construction based on polynomial commitments.

A protocol for zero-testing in accumulated sets:

- To prove that $\vec{v}$ is zero in all indices in set S (for $\vec{v}$ and S respectively committed and accumulated as above, i.e. with underlying polynomial $p_{\vec{v}} := \sum_{i \in [\bar{n}]} v_i \lambda_i(X)$ and $p_S := Z_S(X)$): compute $q(X) := p_{\vec{v}}(X)/p_S(X)$; send polynomial commitment cm_q ; verifier samples a random r ; prover sends $(y_q := q(r), y_v := p_{\vec{v}}(r), y_S := p_S(r), \pi_q, \pi_v, \pi_S)$ where

and rely on verifier-generated random challenges (i.e., the resulting ADS schemes are in the random oracle model). Finally, although it is folklore that vector commitments and accumulators can be derived from polynomial commitments, our technical contribution is to show how *our new properties* can be obtained from polynomial commitments alone.

each $\pi_\star$ is a polynomial evaluation proof for the respective values/polynomials (and might be batched in principle).

- The verifier checks that $y_v = y_q \cdot y_S$ and checks all polynomial evaluation proofs.

The protocol we just presented has proofs consisting of a constant number of polynomial opening (and a constant number of field elements); it is non-interactive and extractable in the ROM using the Fiat-Shamir heuristic if the underlying polynomial commitment is extractable and homomorphic (its security can immediately be reduced to that of polynomial commitments following the standard techniques used in the other proofs in this paper).

The protocol above applies directly to the vector and set commitments described in Section 4. However, in the next paragraphs we also present a pairing-based instantiation that avoids the random oracle.

D.3 An instantiation of zero-testing directly from pairings

The scheme we present now represents a way to perform the same checks as those in Section 5.2, but through the “shortcuts” that pairings allow us. Recall that, in this particular case, the commitment to $\vec{v}$ and the accumulator to S are KZG commitments, that is they are respectively $\text{cm}_{\vec{v}} := [p_{\vec{v}}(\tau)]_1$ and $\text{acc}_S := [p_S(\tau)]_1$ (with τ being the secret element in the setup).

- The prover computes $\pi_{2,S} := [p_S(\tau)]_2$ and $\pi_q := [q(\tau)]_1$ with $q(X) := p_{\vec{v}}(X)/p_S(X)$.
- The verifier performs two checks: (i) $e(\pi_q, \pi_{2,S}) \stackrel{?}{=} e(\text{cm}_{\vec{v}}, [1]_2)$; (ii) $e([1]_1, \pi_{2,S}) \stackrel{?}{=} e(\text{acc}_S, [1]_2)$.

The security of the protocol follows immediately from observations similar to those in the proof of Theorem 4.2.

D.4 The actual compiler

Our final compiler is described in Fig. 13. The executed $(\text{prk}, \text{vrk}) \leftarrow \text{Setup}(1^\lambda)$, is the algorithm defined in Section 4. Notice that this setup is used by all cryptographic tools in our system and can be reused among multiple executions.

$\text{PreProc}(\text{prk}, \text{db})$ is implemented as follows: for each column $\vec{u}_i$ in input to PreProc computes $\text{cm}_i \leftarrow \text{LVC.CommitVec}(\text{prk}, \vec{u}_i)$. For each different value v_i contained in each column a set X_i of the indices where v_i appears in the column (an inverted index) is created. For each set of indices X_i in input computes $\text{acc}_i \leftarrow \text{SA.Accum}(\text{prk}, X_i)$. For each of these accumulators the following pair is computed $(H(v_i), \text{acc}_i)$, where H is a hash function. The output (c, state) is defined as follows: c contains all the cm_i computed together with all the pairs $(H(v_i), \text{acc}_i)$, state contains the additional information for the internal computation. Notice that to have a constant size digest of db , it is possible to create a vector commitment with LVC of all vector commitments and set accumulators.

The algorithms $\text{AnsQry}(\text{prk}, q)$ and $\text{VerQry}(\text{V}, q, a, c, \pi)$ are implicitly reported in the compiler that can be found in Table 13. Notice that every time a set of indices is required, these indices are obtained by asking for the set accumulator associated with the value contained in the column.

Idealized Operation	Cryptographic Implementation
Produce and send new slice handle $\vec{p}$	Send $\text{cm}_{\vec{p}} \leftarrow \text{LVC.CommitVec}(\text{prk}, \vec{v})$
Produce and send new set handle X	Send $\text{acc}_X \leftarrow \text{SA.Accum}(\text{prk}, X)$
$Z \stackrel{?}{=} X \cap Y$	Prover $\pi \leftarrow \text{SA.OpenOp}(\text{prk}, X, Y, \cap)$. Verifier checks $\text{SA.VfOp}(\text{vrk}, \text{acc}_Z, \text{acc}_X, \text{acc}_Y, \cap, \pi)$
$Z \stackrel{?}{=} X \cup Y$	Same as $\cap$, using $\cup$ operator
$X \stackrel{?}{\subseteq} Y$	Prover sends $\pi \leftarrow \text{SA.OpenOp}(\text{prk}, X, Y, \subseteq)$ Verifier checks $\text{SA.VfOp}(\text{vrk}, \text{acc}_X, \text{acc}_Y, \subseteq, \pi)$
$Y \stackrel{?}{=} X^* \setminus X$	Prover sends $\pi \leftarrow \text{SA.OpenOp}(\text{prk}, Y, X, \text{complementOp})$ Verifier checks $\text{SA.VfOp}(\text{vrk}, \text{acc}_Y, \text{acc}_X, \text{complementOp}, \pi)$
$\langle \vec{u}, \vec{v} \rangle \stackrel{?}{=} y$	Prover sends $\pi \leftarrow \text{LVC.OpenLin}(\text{prk}, \vec{u}, \vec{v}, y)$ Verifier checks $\text{LVC.VfLin}(\text{vrk}, \text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, y, \pi)$
$\vec{u} \leftarrow \alpha \vec{v} + \vec{w}$	Uses homomorphism of LVC
$\vec{v} \leftarrow (v_1, \dots, v_n)$	$\text{LVC.CommitVec}(\text{prk}, (v_1, \dots, v_n))$
$\vec{u} [X] \stackrel{?}{=} 0$	Prover sends $\pi \leftarrow \text{LVC.PrSubvec0}^*(\text{prk}, \vec{u}, X)$ Verifier checks $\text{LVC.VfSubvec0}^*(\text{vrk}, \text{cm}_{\vec{u}}, \text{acc}_X, \pi)$
$\text{data} \leftarrow \text{read}(X, \vec{p})$	Prover sends $\pi \leftarrow \text{LVC.OpenSub}(\text{prk}, C, X, \text{data})$ Verifier checks $\text{LVC.VerifySub}(\text{vrk}, C, X, \text{data}, \pi)$ and $\text{acc}_X \stackrel{?}{=} \text{SA.Accum}(\text{prk}, X)$

Figure 13: Compilation of idealized operations through cryptographic building blocks.

D.5 Instantiating our compiler

Here we describe our formal results. The results follow immediately and their proofs follow the common proofs in compilers of this type (e.g., [19, 31]).

Remark about security requirements of the underlying building blocks. While we require extractability as a form of security for our underlying primitives, we do not believe this requirement to be necessary because of the reliance on at least *some* honestly computed commitments. This may allow us to rely on some weaker form *evaluation binding* [57]. We choose to describe our framework in terms of extractability because it makes extremely easy (almost immediate) to prove security of our compilers. Moreover, the extractability properties of KZG—essentially, our core instantiation—have been widely discussed in literature. We leave as future work to a more fine-grained security treatment of our framework.

D.5.1 An instantiation from (augmented) vector commitments & accumulators.

THEOREM D.1. *Let Π_{LVC} and Π_{Acc} be respectively:*

- *an homomorphic linear-map vector commitment scheme (Appendix A.4) with subvector opening (Appendix A.5);*
- *an accumulator with set relation opening (Appendix A.6).*

Let Π_{LVC} and Π_{Acc} also jointly support zero-testing in accumulated sets (Def. 12). Then our compiler (Appendix D.4 and Fig. 13) on input a succinct idealized VDB (Appendix B) produces a succinct VDB (Fig. 6 and Def. 1) supporting the same family of queries.

PROOF. Let V be a succinct idealized VDB as described in Section 5.1 and Appendix B, the protocol V' obtained by the compiler described in Appendix D.4 on input V is a succinct VDB:

Completeness: Intuitively, completeness follows from the completeness of the underlying cryptographic tools. In particular each set handle X is compiled in a set accumulator containing X and each slice handle $\vec{p}$ is compiled in a vector commitment. Every test on set operations is complete for the completeness of the opening operations of set accumulators. The inner product completeness holds for the LVC correctness. The zero test correctness follows from the correctness of the zero-testing in accumulated sets. The new slices from homomorphisms are complete due to homomorphic properties of the LVC commitment. The completeness of the derived operations in Table 2 follows from the correctness of these basic compiled operations.

Soundness: Soundness follows from the extractability of the cryptographic schemes. Since the analysis is extremely standard (e.g., same as in similar compiler approaches as in [19, 31]) we simply provide an intuition. Each proof in the idealized VDB is of the form $\tilde{p}_1^P, X_1^P, \dots, \pi_{\text{hint}}$. We assume this proof is correctly computed while the compiled proof does not satisfy the computational soundness of VDB, i.e., the verification of the proof returns 1 but the response is not correct given the committed database. Let $\mathcal{A}$ be the adversary that breaks the soundness (Fig. 6) with probability greater than negligible. By invoking the extractability of the underlying building block we are able to extract the committed values from $\tilde{p}_1^P, X_1^P, \dots$ sent at proving time (the preprocessing handles are generated honestly and we can directly point to the respective commitment openings in the reduction). That is, we can use the LVC extractors for each slice handle and the set accumulator extractor for each set handle. By the security of the underlying idealized protocol, it holds that the extracted values justify the response with overwhelming probability contradicting the hypothesis that $\mathcal{A}$ can break the soundness.

Succinctness: This follows immediately by inspection: a succinct transcript from an idealized VDB is compiled into a tuple of vector commitments, set accumulator and respective proofs of the same size up to a $O_\lambda(1)$ factor. $\square$

D.5.2 A general instantiation from homomorphic polynomial commitments. When we discussed how to build zero-testing in Section 5.2, we hinted at a connection between polynomials and vector/set commitments. This connection goes beyond zero-testing and can be shown to generalize quite easily to the other functionalities we require in Theorem D.1. To prove the theorem below all we need is to show that we can obtain the ingredients of Theorem D.1 from any polynomial commitment scheme because we can generalize the constructions in the last subsection. This can be observed by inspecting a generalized version of common constructions (Section 4 and Fig. 5) and observing the same type of patterns we noticed in Appendix D.2. (This holds in particular for the specific pairing-based building blocks described in Section 4, which can easily satisfy the requirements of Theorem D.1. We will apply this observation in the next section).

THEOREM D.2. *Let Π_{pc} be a homomorphic extractable polynomial commitment scheme. Then our compiler in Fig. 13 produces, on input a succinct idealized VDB (Appendix B), a succinct VDB (Fig. 6 and Def. 1) supporting the same family of queries in the random oracle model (ROM).*

PROOF. We have to show that all the operations guaranteed by Π_{LVC} and Π_{Acc} are implied by any Π_{pc} .

Therefore, we can show how to perform each operation of Π_{LVC} and Π_{Acc} using polynomial evaluations. For Π_{LVC} :

- For the operations $\text{OpenLin}(\text{prk}, \vec{u}, \vec{v}, y)$ and $\text{VfLin}(\text{vrk}, \text{cm}_u, \text{cm}_v, y, \pi)$, the prover and the verifier interact as follows: the verifier receives from the prover the commitments $\text{cm}_{\vec{u}},$

$\text{cm}_{\vec{v}}, \text{cm}_{R(x)}, \text{cm}_{H(x)}, \text{cm}_{\hat{R}(x)}$ such that $\hat{R}(x) = x^2 R(x)$, $\deg(R) < \bar{n} - 1$ and

$$\left(\sum_{i=1}^{\bar{n}} u_i \lambda_i(x) \right) \left(\sum_{i=1}^{\bar{n}} v_i \lambda_i(x) \right) - \bar{n}^{-1} y = x R(x) + t(x) H(x). \quad (1)$$

The verifier sends a random value k to the prover. The prover sends $\pi_{\phi_j} \leftarrow \text{OpenPos}(\text{prk}, \phi_j(k), k, d)$, where $\phi_j \in \{\vec{u}, \vec{v}, R(x), H(x), \hat{R}(x)\}$. The verifier checks that $\text{VfPos}(\text{vrk}, \text{cm}_j, k, \phi_j(k), \pi_{\phi_j}) = 1$ for all $\text{cm}_j \in \{\text{cm}_{\vec{u}}, \text{cm}_{\vec{v}}, \text{cm}_{R(x)}, \text{cm}_{H(x)}, \text{cm}_{\hat{R}(x)}\}$ and evaluates Eq. (1) at $x = k$. If the opening proofs are valid, the equations $\hat{R}(k) - k^2 R(k) = 0$ and Eq. (1) hold, the Schwartz-Zippel lemma guarantees that the proof holds.

- $\text{OpenSub}(\text{prk}, \text{aux}_{\vec{u}}, I)$ and $\text{VerifySub}(\text{vrk}, \text{cm}_u, \vec{v}, I, \pi)$ can be computed using the same strategy of $\text{OpenLin}(\text{prk}, \vec{u}, \vec{v}, y)$ and $\text{VfLin}(\text{vrk}, \text{cm}_u, \text{cm}_v, y, \pi)$.

For the set accumulator, the fact that we can construct them from polynomial commitments has already been shown through the description in Fig. 5. The zero-testing in accumulated sets follows the blueprint described in Appendix D.3. To make the protocol non interactive the prover can use the Fiat-Shamir heuristic producing the challenges using a RO. $\square$

E Additional Details on How qedb Works

The following list contains the most relevant types of queries that are supported by our construction. Two of them (conjunctive and JOIN queries) are also described in Section 6.2.

E.1 Simple selection queries.

We provide some examples of simple queries over tabular data. We start considering a single table T composed of several columns $\text{col}_1, \text{col}_2, \dots, \text{col}_u$.

The simplest query returns some column from T , without any selection condition (in this case the Prover does not have to check any selection condition):

SELECT $\text{col}_1, \text{col}_2$ FROM T

Pre-processing: The Data Owner defines slice handles $\mathbb{P}_{\text{col}_1}, \mathbb{P}_{\text{col}_2}$

Proof computation: Prover does nothing.

Proof verification: Verifier retrieves the data in columns $\text{col}_1, \text{col}_2$ with $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_1})$ and $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_2})$.

Completeness holds since the Verifier collects the handles $\mathbb{P}_{\text{col}_1}$ and $\mathbb{P}_{\text{col}_2}$ to col_1 and col_2 from T_{handles} and $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_1})$ and $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_2})$ return exactly what prescribed by the calls to $\mathcal{O}^{\text{read}}(\mathbb{P}_{\text{col}_1}, \mathbb{O}^*)$ and $\mathcal{O}^{\text{read}}(\mathbb{P}_{\text{col}_2}, \mathbb{O}^*)$. Soundness holds since the Verifier is not taking anything from the Prover and only computes $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_1})$ and $\text{read}(\mathbb{O}^*, \mathbb{P}_{\text{col}_2})$. These commands return exactly the values contained in col_1 and col_2 , therefore $\text{resp} \in Q(q, \text{db})$.

The following query contains an equality-based selection condition:

SELECT col₁ FROM T WHERE col₂ = u

Pre-processing: we assume that the Data Owner knows the set U to which the value u belongs to. The Data Owner computes the slice handle $\mathcal{f}col_1$; then it computes an *inverted index* associated to the set U: (i) for all the values of U appearing in col₂, take note of all their positions in such column; (ii) compute set handles of such lists of positions. More formally, for each $u \in U$ define the inverted index $X_u = \{j : col_2[j] = u\}$. Then, define the set handle $\mathcal{X}_u$.

Proof computation: As in the previous query, the Prover does nothing.

Proof verification: The Verifier picks $\mathcal{X}_u$ and outputs $read(\mathcal{X}_u, \mathcal{f}col_1)$.

Completeness holds since the Verifier collects the handles $\mathcal{f}col_1$ to col₁ from $\mathcal{T}^{handles}$ and $\mathcal{X}_u$ from $\mathcal{T}^{handles}$, therefore the Verifier computes $read(\mathcal{X}_u, \mathcal{f}col_1)$ that returns exactly the positions in col₁ where col₂ = u that is what prescribed by the calls to $O^{read}(\mathcal{f}col_1, \mathcal{X}_u)$. Soundness holds since the Verifier is not taking anything from the Prover and only computes $read(\mathcal{X}_u, \mathcal{f}col_1)$. This command returns exactly the values contained in col₁ required by the query, therefore $resp \in Q(q, db)$.

E.2 Range queries and conjunctive queries

Consider the following range query:

SELECT col₁ FROM T WHERE col₂ $\in [\alpha, \beta]$

Pre-processing: as before.

To show that X_{in} is the set of all indices j such that $\alpha \leq \vec{v}[j] \leq \beta$, Prover and Verifier execute the following steps (we have renamed col₂ as $\vec{v}$):

- Prover sends $\mathcal{X}_{out}$
- Verifier checks that all the values indexed by X_{in} are in the range:
- Prover sends $\vec{u}_1$, where $\vec{u}_1$ is equal to 1 for each index in X_{in} and 0 elsewhere
- Verifier checks that $\vec{u}_1 \stackrel{?}{=} \vec{1} \left[\mathcal{X}_{out} \rightarrow 0 \right]$
- let $\vec{u}_{\geq \alpha} := \vec{v} - \alpha \vec{u}_1$
- let $\vec{u}_{\leq \beta} := \beta \vec{u}_1 - \vec{v}$
- Prover and Verifier run the protocol to check that $\vec{u}_{\geq \alpha} \geq 0$
- Prover and Verifier run the protocol to check that $\vec{u}_{\leq \beta} \geq 0$
- // check that no value out of those indexed by X_{in} is in the range:
- Prover sends two additional set handles $\mathcal{X}_{<\alpha}$, $\mathcal{X}_{>\beta}$
- Verifier checks that $\mathcal{X}_{in}$, $\mathcal{X}_{<\alpha}$, $\mathcal{X}_{>\beta}$ form a partition of $\mathcal{X}_{out}$
- let $\vec{u}_{<} := \alpha \vec{u}_1 - \vec{v}$
- let $\vec{u}_{>} := \vec{v} - \beta \vec{u}_1$

- Prover and Verifier run the protocol to check that

$$\vec{u}_{<} [\mathcal{X}_{<\alpha}] \geq 0$$

- Prover and Verifier run the protocol to check that

$$\vec{u}_{>} [\mathcal{X}_{>\beta}] \geq 0$$

- Verifier performs $read(\mathcal{X}_{in}, \mathcal{f}col_1)$

Completeness holds since for each response that is correct, thanks to the correctness of the underlying scheme, the Prover can correctly show that elements indexed in X_{in} are in the range, while the others values are outside the range. Let us assume that soundness does not hold, therefore there is a Prover that generates accepting response resp such that $resp \notin Q(q, db)$. Let $\pi_\alpha, \pi_\beta, \pi_{<}, \pi_{>}$ be the check procedure of – respectively – the chosen values in col₂ are contained in the range $[\alpha, \beta]$, and no other values have been chosen. Since π_α guarantees that all the elements in col₂ in positions contained in X_{in} are greater or equal to α , π_β guarantees that values are also lower than or equal to β . The first Verifier check guarantees that there is no intersection among the two sets X_{in} and X_{out} . The proof $\pi_{<}$ guarantees that the elements in $X_{<}$ are all small than α while $\pi_{>}$ shows that elements in $X_{>}$ are all greater than β . Therefore $read(\mathcal{X}_{in}, \mathcal{f}col_1)$ is equal to resp and $resp \in Q(q, db)$.

As for general conjunctive query, consider the query:

SELECT col₁ FROM T WHERE $Test_1(col_2)$ AND $Test_2(col_2)$

Pre-processing: as before.

Proof computation: the Prover computes the set handles $\mathcal{X}_{Test_1}$, $\mathcal{X}_{Test_2}$ of the indices for which $Test_1$ and $Test_2$ respectively hold and sends them to the Verifier.

Proof Verification: the Verifier performs the following steps:

- after renaming the set handles received from the Prover to $\mathcal{X}_{in,1}$, $\mathcal{X}_{in,2}$ respectively, define the set handle $\mathcal{X}_{in,\wedge} = \mathcal{X}_{in,1} \cap \mathcal{X}_{in,2}$
- output $read(\mathcal{X}_{in,\wedge}, \mathcal{f}col_1)$

Completeness follows from the the fact that $Test_1(col_2)$ AND $Test_2(col_2)$ will return the correct set handles $\mathcal{X}_{Test_1}$ and $\mathcal{X}_{Test_2}$. Consequently, the computation of $\mathcal{X}_{in,\wedge}$ is also correct, and therefore $read(\mathcal{X}_{in,\wedge}, \mathcal{f}col_1)$ outputs all values in col₁ satisfying both tests. Soundness holds since the only data sent by the Prover are $\mathcal{X}_{Test_1}$ and $\mathcal{X}_{Test_2}$. Let us assume that there exists an adversarial Prover that is able to return values $\mathcal{X}_{Test_1}$ and $\mathcal{X}_{Test_2}$ such that the intersection among these indexes contains wrong elements. Let $\mathcal{X}_{Test_1}$ and $\mathcal{X}_{Test_2}$ be correct values for the soundness of $Test_1(col_2)$ and $Test_2(col_2)$. Since the read is computing the elements obtained by the intersection of $\mathcal{X}_{Test_1}$ and $\mathcal{X}_{Test_2}$ and the intersection algorithm is sound, it holds that $resp \in Q(q, db)$.

E.3 Join queries

Consider tables T_1, T_2 with respective columns named pk, col_1 and fk, col_2 . As their names suggest pk is primary key of table T_1 , and fk is a foreign key in T_2 referencing values from pk . Consider the query:

```
SELECT * FROM T1 JOIN T2 ON pk = fk
```

Pre-processing: as before.

Proof computation: the Prover performs the following steps:

- retrieves the set handle $\mathcal{J}fk$ referring the inverse lookup $\{j : fk[j] = v\}$, for each $v \in V_{pk}$.
- retrieves the set handle $\mathcal{J}pk$ referring the inverse lookup $\{j : pk[j] = v\}$, for each $v \in V_{fk}$.

The Prover sends $\mathcal{J}pk, \mathcal{J}fk$ to the Verifier.

Proof verification: The Verifier performs the following steps:

- compute $\widehat{pk} \leftarrow \text{read}(\mathcal{J}pk, \mathcal{J}pk)$
- compute $\widehat{fk} \leftarrow \text{read}(\mathcal{J}fk, \mathcal{J}fk)$
- check that $\widehat{fk} = \widehat{pk}$
- $\widehat{rst}_1 \leftarrow \text{read}(\mathcal{J}pk, T_1.rst)$
- compute $\widehat{rst}_2 \leftarrow \text{read}(\mathcal{J}fk, T_2.rst)$

Subsequently, the Verifier concatenates the data retrieved in $\widehat{pk}, \widehat{rst}_1$ and $\widehat{rst}_2$. To prove that the query result contains all the valid tuples, Prover and Verifier engage in a protocol similar to the second part of the one defined for Query E.2.

To join two tables T and T' on equality on column C with duplicates in both tables, we do the following:

Invariant (initially enforced through indexing):

- For each table T and column C we keep a set of values $V(T, C)$ in that column

Observation: let $V_\cap := V(T, C) \cap V(T', C) = \{\alpha_1, \dots, \alpha_\ell\}$, the JOIN will be given by the cross product of the rows from each table as follows:

$$\alpha_i^{-1}(T.C) \times \alpha_i^{-1}(T'.C) \text{ for } i = 1, \dots, \ell$$

where recall that $\alpha^{-1}(\vec{v}) := \{j : v_j = \alpha\}$ denotes the set of preimages of the value α in the vector $\vec{v}$.

Our protocol is then as follows:

- the Prover sends $V_\cap$ as defined above to the Verifier (in the clear—this is not a handle)
- it also sends for each $\alpha \in V_\cap$, the respective preimage set (as a handle) in each table. I.e., it sends $\mathcal{X}_{\alpha, \text{col}} := \alpha^{-1}(\text{col})$ for $\text{col} \in \{T.C, T'.C\}$ and $\alpha \in V_\cap$.
- Then, for each $\mathcal{J}\text{col} \in \{\mathcal{J}T.C, \mathcal{J}T'.C\}$ and $\alpha \in V_\cap$, the Prover and Verifier run a sub-protocol $X_{\alpha, \text{col}} \stackrel{?}{=} \alpha^{-1}(\mathcal{J}\text{col})$.
- They also run a sub-protocol to show $\mathcal{Y}_\cap \stackrel{?}{=} \mathcal{Y}(T.C) \cap \mathcal{Y}(T'.C)$ (recall that the last two handles are already held by the Verifier as handles from the invariant).

- Finally the Verifier runs $\text{read}(\cup_\alpha \mathcal{X}_{\alpha, T.C}, \mathcal{J}T.\star)$ and $\text{read}(\cup_\alpha \mathcal{X}_{\alpha, T'.C}, \mathcal{J}T'.\star)$ (i.e., it reads any column selected for the JOIN) and performs the cartesian product appropriately.

The protocol above directly extends to multi-way JOINS.

Completeness follows since the Prover is honestly computing $V_\cap$ and for each element $\alpha_i \in V_\cap$ the correct $\mathcal{X}_{\alpha, \text{col}}$. By the correctness of the preimage check protocol (see Table 2), the Verifier accepts the proofs for all $\mathcal{X}_{\alpha, \text{col}}$ with overwhelming probability. By the correctness of the intersection protocol, the Verifier accepts the proof that $V_\cap = V(T, C) \cap V(T', C)$ with overwhelming probability. By the completeness of the read protocol, the Verifier receives with overwhelming probability all rows in which the join columns contain values from $V_\cap$. Finally, the local computation of the cartesian product for each matching value $\alpha \in V_\cap$ produces exactly the correct JOIN result. Therefore, completeness holds with probability negligibly close to 1.

For soundness, let us assume that there exists an idealized adversary $\mathcal{A}$ that causes the Verifier to accept an incorrect response resp. resp can be incorrect for one of the following scenarios: (i) $V_\cap$ is incorrect and omits some $\alpha \in V(T, C) \cap V(T', C)$. By the soundness of the subset protocol, we have that if $S \neq S_T \cap S_{T'}$, then the subset verification is rejected. This contradicts the assumption that the verification is successful. (ii) $V_\cap$ contains an α that does not belong to $V(T, C) \cap V(T', C)$. If $\alpha \notin V(T, C)$, then either the preimage set $X_{\alpha, T.C}$ is empty, or $\mathcal{A}$ stores in $V_\cap$ false positions j where $T.C[j] \neq \alpha$. In this case, the preimage check fails. Indeed, the equality test of the preimage check, i.e., $X_{\alpha, T.C} \stackrel{?}{=} \alpha^{-1}(\mathcal{J}T.C)$, detects $\text{col}[j] \neq \alpha$ for $j \in X_{\alpha, T.C}$. The same argument applies if $\alpha \notin V(T', C)$. (iii) For some $\alpha \in V_\cap$, the Prover sends $X'_{\alpha, \text{col}} \neq \alpha^{-1}(\text{col})$ where $X'_{\alpha, \text{col}}$ contains position j with $\text{col}[j] \neq \alpha$. The test $\mathcal{X}_{\alpha, \text{col}} \stackrel{?}{=} \text{eqSet}(\vec{\mu}_\alpha, (\mathcal{J}\text{col}))$ checks that the two slices are equal in positions in $X_{\alpha, \text{col}}$ and different in the other positions. If $\text{col}[j] \neq \alpha$ for some $j \in X'_{\alpha, \text{col}}$, this check fails. (iv) For some $\alpha \in V_\cap$, the set $X'_{\alpha, \text{col}}$ omits position j where $\text{col}[j] = \alpha$. The preimage check protocol ensures that all positions with value α are included. Specifically, it verifies that $\text{col}[j] = \alpha$ for all $j \in X_{\alpha, \text{col}}$, and that $\text{col}[j] \neq \alpha$ for all $j \notin X_{\alpha, \text{col}}$. If position j is omitted but $\text{col}[j] = \alpha$, then j is in the complement set. Therefore, with overwhelming probability the Verifier rejects the proof. (v) The read operation returns wrong data, but for the soundness of the underlying protocol it cannot happen.

By combining all cases, if all verification checks pass, it holds that $\text{resp} \in Q(q, \text{db})$. This contradicts the assumption that resp is incorrect.

E.4 Aggregate queries

We now present the protocols for min, sum, count, and average queries.

- MIN query: consider the query:

```
SELECT MIN(coltgt) FROM T
```

Pre-processing: we assume that the Verifier has the slice handle $\mathbb{t}_{tgt}$ corresponding to col_{tgt} .

Proof computation: the Prover does as follows:

- compute the set of positions $argmin(tgt)$ in col_{tgt} of the minimum values: $argmin(tgt) = \{j : col_{tgt}[j] \leq v \text{ for all } v \in col_{tgt}\}$ (here we denote with $col_{tgt}[j]$ the j -th element in the slice referred by $\mathbb{t}_{tgt}$).
- compute $\mathcal{X}_{argmin}$ and sends it to the Verifier

Proof verification: the Verifier performs the following steps:

- get $\mathcal{X}_{argmin}$
- retrieve $v_{min} \leftarrow \text{read}(\mathcal{X}_{argmin}, \mathbb{t}_{tgt})$
- define $\mathbb{t}_{tgt}' = \mathbb{t}_{tgt} - \mathbb{1}(v_{min}, \dots, v_{min})$
- check that every value in the slice handle $\mathbb{t}_{tgt}'$ is non-negative.

Notice that the completeness of this procedure is trivial since it is enough that the Verifier can obtain a single minimum value.

Soundness holds since a dishonest Prover cannot claim a smaller value, because no such value appears in the column and the read check fails, nor a larger value, because the range check would fail for the true minimum row. I.e., in both cases $\mathbb{t}_{tgt}'$ contains at least one negative number.

The protocol for max is a trivial adaptation of the above.

- SUM query: consider the query:

```
SELECT SUM(coltgt) FROM T
```

Pre-processing: as before.

Proof computation:

- the Prover computes the set handle $\mathcal{S}^*$ corresponding to the rows of T and the value $s_{tgt} = \sum_{v \in col_{tgt}} v$
- The Prover sends $\mathcal{S}^*$ and s_{tgt} to the Verifier

Proof Verification:

- the Verifier gets $\mathcal{S}^*$ and s_{tgt} from the Prover
- the Verifier checks that $\sum_{\mathcal{S}^*} \mathbb{t}_{tgt}$ is equal to s_{tgt}

Completeness follows from the correctness of the sum check within target subset operation, indeed the Prover is sending $\mathcal{S}^*$ together with $\vec{u}_1$, I.e., the vector that contains all ones, the Verifier checks that $\vec{u}_1$ is actually one in all positions and then performs the inner product between col_{tgt} and $\vec{u}_1$ checking if it is equal to the response. To prove soundness let us assume that there exists an adversary $\mathcal{A}$ that causes the Verifier to return 1 but such that $\text{resp} \notin Q(q, db)$. Therefore resp does not contain the sum of the elements in col_{tgt} . The proof generated by $\mathcal{A}$ cannot be accepted by the Verifier, since the Verifier can check that $\vec{u}_1$ is a vector of all ones, that $\mathcal{S}^*$ is indeed a set handle to all indices and that the inner product of the two handles is actually the expected sum.

- COUNT query: consider the query:

```
SELECT COUNT(coltgt) FROM T
```

Pre-processing: as before.

Proof computation:

- the Prover computes the set handle $\mathcal{S}^*$ referring to all rows of T
- the Prover sends $\mathcal{S}^*$ and the value n to the Verifier

Proof Verification: The Verifier performs the following steps:

- get $\mathcal{S}^*$ and the value n from the Prover,
- define slice handle $\mathbb{ones} \leftarrow \underbrace{(1, \dots, 1)}_n$
- check that $\sum_{\mathcal{S}^*} \mathbb{ones}$ is equal to n

Completeness and soundness discussion are equal to the one done for Q7.

E.5 Nested queries

Consider the query:

```
SELECT col1 FROM T1 WHERE col2 IN  
(SELECT col3 FROM T2 WHERE col4 = u)
```

Pre-processing: as before.

Proof computation: Prover sends to Verifier the following set handles: $\mathcal{S}^{col_{4u}}$ (the entry of set handle $\mathcal{S}^{col_4}$ corresponding to value u)

and $\mathcal{S}^{col_{2v_1}}, \mathcal{S}^{col_{2v_2}}, \dots, \mathcal{S}^{col_{2v_n}}$, that are the set handles entries in $\mathcal{S}^{col_2}$ of the values $\{v_1, v_2, \dots, v_n\}$ contained in the answer to the sub-query $\text{SELECT } col_3 \text{ FROM } T_2 \text{ WHERE } col_4 = u$.

Proof verification: Verifier creates the new set handle $\mathcal{S}^{col_{2OR}}$ equal to $\mathcal{S}^{col_{2v_1}} \cup \mathcal{S}^{col_{2v_2}} \cup \dots \cup \mathcal{S}^{col_{2v_n}}$; finally, Verifier retrieves the answer of the nested query via $\text{read}(\mathcal{S}^{col_{2OR}}, \mathcal{S}^{col_4})$.

Again, to prove that the query result contains all the valid tuples, Prover and Verifier engage in a protocol similar to the second part of the one defined for Q5. In general, we can handle nested queries through techniques analogous to those in [96] but without having to rely on a preprocessing containing auxiliary info on every possible pair of columns in the same table (which leads to its quadratic blowup).

E.6 Group by queries

Group by queries (Simple). Consider the query, where the condition on $\text{agg}(col_{tgt})$ can be any aggregation expression.

```
SELECT col1, agg(coltgt) FROM T1 GROUP BY col1
```

Pre-processing: We assume that the Verifier has the slice handle $\mathbb{t}_{tgt}$ corresponding to col_{tgt} together with the set handles for column col_1 .

Proof computation: the Prover performs the following steps:

- select the elements set $\{\alpha_1, \dots, \alpha_n\}$ in col_1 corresponding to the indices in X_{col_2}
- given the values $\{\alpha_1, \dots, \alpha_n\}$ in col_1 , compute the sets of indices $X_1, \dots, X_n$ where $X_i = \{j : \text{col}_1[j] = \alpha_i\}$
- compute the aggregations $a_1, \dots, a_n$ on col_{tgt} on indices $X_1, \dots, X_n$
- send $X_1, \dots, X_n$ to the Verifier
- the Prover and the Verifier run a sub-protocol to prove that $X_1 \cup \dots \cup X_n = \mathcal{O}^*$
- the Verifier checks that each X_i is contained in the set accumulator for column col_1
- the Prover and the Verifier run a sub-protocol to prove that each aggregation a_i , for $i \in [n]$ is correct in tgt under the indices X_i .

Let us assume that completeness does not hold, in this case either the Prover did not send a pair (α_j, a_j) or there exists at least a value $\alpha_j \in \{\alpha_1, \dots, \alpha_n\}$ such that the aggregator is not aggregating all values. In the first case it implies that $X_1, \dots, X_n$ is not a partition of $*$ (i.e., proving that each X_i is in a set handle for col_1 and that the union of all X_i is equal to $\mathcal{O}^*$) that is a contradiction. The second case implies that the aggregation proof is not complete statement that is a contradiction. Let us assume that soundness does not hold. In this case, either in (α_j, a_j) , a_j does not correspond to the indices in X_j or there exists an α_j such that is not in the positions X_{col_2} . The first case contradicts the soundness of the underlying protocol. The second case instead violates one of the checks performed by the Verifier in the containment of X_i for column col_1 .

Group by queries (with Filter). Consider the query, where the condition on $\text{agg}(\text{col}_{tgt})$ can be any aggregation expression and the condition on col_2 can be any filter predicate.

```
SELECT col1, agg(coltgt) FROM T1
WHERE cond(col2) GROUP BY col1
```

Pre-processing: we assume that the Verifier has the slice handle tgt corresponding to col_{tgt} together with the set handles for column col_1 , the slice handle col_2 and set handles corresponding to col_2 .

Proof computation: the Prover performs the following steps:

- compute the set of positions X_{col_2} in col_2 that satisfy the filter predicate
- select the elements set $\{\alpha_1, \dots, \alpha_n\}$ in col_1 corresponding to the indices in X_{col_2}
- compute the sets of indices $X_1, \dots, X_n$ where $X_i = \{j : \text{col}_1[j] = \alpha_i\}$
- compute the aggregations $a_1, \dots, a_n$ on col_{tgt} on indices $X_1, \dots, X_n$
- send $X_{\text{col}_2}, X_1, \dots, X_n$ to the Verifier
- the Prover and the Verifier run a sub-protocol to check that X_{col_2} is the set of indexes that satisfy the filter predicate

- the Prover and the Verifier run a sub-protocol to prove that $X_1 \cup \dots \cup X_n = X_{\text{col}_2}$
- the Verifier checks that each X_i is contained in the set accumulator for column col_1
- the Prover and the Verifier run a sub-protocol to prove that each aggregation a_i , for $i \in [n]$ is correct in tgt under the indices X_i .

Let us assume that completeness does not hold, in this case either the Prover did not send a pair (α_j, a_j) or there exists at least a value $\alpha_j \in \{\alpha_1, \dots, \alpha_n\}$ such that the aggregator is not aggregating all values. In the first case it implies that $X_1, \dots, X_n$ is not a partition of X_{col_2} (i.e., proving that each X_i is in a set handle for col_1 and that the union of all X_i is equal to X_{col_2}) which is a contradiction. The second case implies that the aggregation proof is not complete statement that is a contradiction. Let us assume that soundness does not hold. In this case, either in (α_j, a_j) , a_j does not correspond to the indices in X_j or there exists an α_j such that is not in the positions X_{col_2} . The first case contradicts the soundness of the aggregation proof. The second case instead violates one of the checks performed by the Verifier in the containment of X_i for column col_1 .

Group by queries (Group by Multiple Columns). Note that if it is needed to group by multiple columns with a query as

```
SELECT col1, col2 agg(coltgt) FROM T1
WHERE cond(col3) GROUP BY col1, col2
```

for the elements $\{\alpha_1, \dots, \alpha_n\}$ in col_1 the Prover computes $X_1, \dots, X_n$ of the indices where $X_i = \{j : \text{col}_1[j] = \alpha_j\}$ and for the elements $\{\beta_1, \dots, \beta_m\}$ in col_2 the Prover computes $Y_1, \dots, Y_m$ of the indices where $Y_i = \{j : \text{col}_2[j] = \beta_j\}$. Moreover, the Prover computes $Z_{1,1}, \dots, Z_{1,m}, Z_{2,1}, \dots, Z_{2,m}, \dots, Z_{n,1}, \dots, Z_{n,m}$, where $Z_{i,j} = X_i \cap Y_j$. The Prover sends $X_i, Y_j, Z_{i,j}$, for all $i \in [n]$ and $j \in [m]$, to the Verifier. They run a protocol to prove that each $Z_{i,j} = X_i \cap Y_j$, and that $X_i = Z_{i,1} \cup \dots \cup Z_{i,m}$.

E.7 Other queries

Our construction takes into account queries that contain predicates and expressions involving two or more columns. Consider the query:

```
SELECT c1 FROM T WHERE c2 ≥ 2c3 + c4
```

This can be handled immediately thanks to the homomorphic properties of our framework (for simplicity here we omit the usual color coding for handles). For example, in the above the Prover can first homomorphically compute a “virtual” column (a slice) $c' := c_2 - 2c_3 - c_4$ (and so can the Verifier) and then provide a set handle of rows in which such a column would have a value greater than 0 (as we did for the simple case of range queries). The Verifier can then read that set from c_1 . Completeness and soundness discussion are equal to the one done for Q3.

Similarly, our system can handle queries that involve retrieving combinations of columns, e.g., $\text{SELECT } 3c_1 + c_5 \text{ WHERE } [\dots]$. Here

the Prover and Verifier can produce a new column c' (in this case $c' := 3c_1 + c_5$). The read will occur on this specific column. For more details on homomorphisms in our framework we also refer the reader to Fig. 8 and corresponding section.

F Queries Supported in qedb

Queries supported by both this work and IntegriDB:

SELECT with different types of predicates (see below).

Predicate types: Multi-dim. range queries, list membership, any AND/OR.

Aggregate queries: MAX, MIN, COUNT, SUM, AVG.

GROUP BY queries.

JOINS: Equality-based joins over columns, possibly with duplicates.

Nesting of queries.

Queries supported by this work but not by IntegriDB:

Comparison between columns: Predicates involving more than one column (e.g. “[...] WHERE $c_1 \geq 2c_2 + c_3$ ”).

Aggregation among columns: Expressions involving more than one column in the SELECT clause (e.g. “SELECT $c_1 + 2c_2$ FROM [...]”).

Figure 14: Query types supported in IntegriDB (top) vs our work (top and bottom).

G Detailed results

DB size	Q_{Tot}		Q_{CntTx}		Q_{Match}	
	P	V	P	V	P	V
2^{12}	0.12	0.0067	0.61	0.0134	0.07	0.0039
2^{14}	0.39	0.0067	1.95	0.0134	0.26	0.0052
2^{16}	1.46	0.0068	7.03	0.0136	0.80	0.0104
2^{18}	8.36	0.0068	35.63	0.0136	2.89	0.0347
2^{20}	83.69	0.0068	291.37	0.0139	10.48	0.1259

Table 3: Experimental evaluation of query performance for Q_{Tot} , Q_{CntTx} , and Q_{Match} presented in Fig. 2, evaluated over increasing database sizes. All times are reported in seconds.

H Background on Multivariate Sumcheck and GKR

Here we describe the GKR protocol [46] for layered circuits and its core building block, the sumcheck protocol.

The Sumcheck Protocol. The Sumcheck protocol is an interactive proof that lets a verifier check the value of a large exponential sum of a multivariate polynomial using only polynomial effort. Given a

polynomial $f : \mathbb{F}^m \rightarrow \mathbb{F}$ of degree at most d in each variable, and a subset $\mathbb{B} \subseteq \mathbb{F}$, the goal is to verify a claimed value

$$\beta = \sum_{b_1, \dots, b_m \in \mathbb{B}} f(b_1, \dots, b_m).$$

The protocol proceeds in m rounds, one for each variable. In round i , the prover sends a univariate polynomial g_i that is the partial sum of f with the first $i - 1$ variables fixed. The verifier checks that g_i has a degree at most d and that it is consistent with the previous round. The verifier then sends a random challenge $t_i \in \mathbb{F}$, fixing the i -th variable for the next round. In the final round, the verifier evaluates f directly on a single random point to complete the check.

If the prover is honest, all checks pass. If the claimed sum is false, then with probability at least $1 - dm/|\mathbb{F}|$ the verifier detects an inconsistency. The protocol uses m rounds, with communication $O(dm \log |\mathbb{F}|)$, and gives the verifier an efficient way to check a sum over exponentially many inputs.

The final protocol for layered circuits. Now we describe the GKR interactive proof for verifying the correct evaluation of a layered circuit of size S and depth D . The key idea is to iteratively reduce claims about values of gates in higher layers of the circuit to claims about gates in lower layers, eventually reaching the input layer where the verifier can check correctness directly.

Consider an arithmetic circuit with fan-in 2. Each gate in a layer is assigned a unique label from a domain $\mathbb{B}^m$, where typically $\mathbb{B} = \{0, 1\}$ or a similar set, and functions $V_i : \mathbb{B}^m \rightarrow \{0, 1\}$ map labels to gate values in layer i . Their low-degree extensions $\tilde{V}_i : \mathbb{F}^m \rightarrow \mathbb{F}$ are then defined over a larger field $\mathbb{F}$.

The protocol runs in D stages. At stage i , the prover claims that $\tilde{V}_i(z_i) = v_i$ for some point z_i . Using a reduction, this claim is expressed in terms of gate values in the next layer $i + 1$, involving the low-degree extensions of indicator functions that encode the wiring of the circuit (namely, whether a gate is an addition or multiplication of its children). This expression resembles a large sum, which can be efficiently verified using the sumcheck protocol. At the end of the reduction, the verifier is left with a claim about values in layer $i + 1$.

By repeating this process across all layers, the protocol eventually reduces the verifier’s task to checking an evaluation of $\tilde{V}_D$ at some point z_D , where V_D encodes the input layer. Since the input bits are known, the verifier can compute this check directly. Thus, if the prover misreports the output, the verifier catches the inconsistency with high probability while performing only polylogarithmic work in the size of the circuit.

I Typical Building Blocks in zkVMs

In this appendix, we provide additional context on the internal complexity of zkVM-based approaches, which is relevant to our comparison in Section 7.4. While zkVMs offer flexibility by allowing arbitrary computations to be proven, this generality comes at the cost of significant architectural complexity.

Overview of zkVM Architecture. A zkVM (zero-knowledge Virtual Machine) proves the correct execution of a program by: (i) defining an abstract CPU with a fixed instruction set; (ii) executing the program and recording a trace of all state transitions; (iii) proving

that the trace is consistent with the CPU’s semantics. This approach moves away from manually writing circuits for each computation, but the underlying proving machinery remains intricate.

Core Proving Components. The proving backend of a typical zkVM involves several layers:

- *SNARK backends:* Most zkVMs rely on one or more general-purpose SNARKs. Common choices include Groth16 [47] (for its small proof size, often used in the final compression step) and STARK-like systems. The latter involves: arithmetization of the computation into polynomial constraints (often using AIR—Algebraic Intermediate Representation), low-degree testing via FRI (Fast Reed-Solomon Interactive Oracle Proofs), and careful handling of boundary constraints to link consecutive execution steps.
- *Recursive proving:* To handle large computations, zkVMs typically segment the execution trace into chunks, prove each chunk independently, and then recursively aggregate these proofs. This requires circuits that verify proofs within the proving system itself—a technically demanding component that adds substantial code complexity.
- *Constraint systems for the VM:* The CPU abstraction must be encoded as a constraint system. This includes constraints for: instruction decoding, memory read/write consistency, register updates, and control flow. Even for a minimal instruction set, this encoding spans thousands of constraints.

Additional Infrastructure. Beyond the cryptographic core, zkVMs require:

- *Trace segmentation logic:* Determining how to split a computation into segments that fit within prover memory constraints.
- *Parallelization and orchestration:* Coordinating multiple provers (often across GPUs or distributed systems) to handle segments concurrently.

Lines of code in representative zkVMs. For the popular zkVM SP1 [86], we found approximately 230K lines of code, of which 100K in Rust. In addition to these, this system (like several other zkVMs) also requires the proving backend in Plonky3 [77], which alone contains ≈ 100 K lines of code.

Jolt. The zkVM Jolt has a different design than most zkVMs. Its architecture is arguably simpler in blueprint and implementing the vision of the *lookup singularity*:) it relies heavily on lookup arguments to efficiently emulate CPU operations. Jolt uses the Lasso lookup argument to decompose CPU instruction execution into table lookups. We measured at least 80K Rust lines of code in Jolt [1]. This is a lower bound which excludes dependencies such as Dory.

Contrast with qedb. Crucially, in a zkVM-based VDB, the SQL query logic must be layered on top of all the components described above—either compiled to a circuit or executed as a program whose trace is then proven. In qedb, database operations are directly represented through our idealized protocol abstractions (Section 5.1), which compile to basic cryptographic building blocks without intermediate layers. This directness is a key source of both our simplicity. The total number of lines of code for cryptographic components is approximately ≈ 8 K.

J More on the Security of zkVMs

Here we expand on what we mentioned in Section 7.4 as some of the delicate aspects around the security of zkVMs.

- *Aggressive coding-theoretic conjectures.* Many zkVMs rely internally on STARK-based proof systems, whose most efficient instantiations depend on strong conjectures about the distance properties of certain codes. Some of these conjectures have recently been shown to be false [35].
- *Recursion depth.* zkVMs typically employ recursive proof composition to handle large computations. However, how the security of recursive proof systems degrades with recursion depth remains an open problem; in most cases, we lack proofs of security for arbitrary depths [17, 18, 29, 61, 67].
- *Fiat-Shamir and the random-oracle model.* These constructions rely heavily on the Fiat-Shamir heuristic, which is analyzed in the random-oracle model. Yet, to enable recursion, the same hash function must be expressed as a concrete arithmetic circuit—an assumption that is formally inconsistent with modeling the hash as a random oracle. The security implications of this tension are not well understood. This concern is particularly salient in light of recent work demonstrating practical attacks on Fiat-Shamir in real-world proof systems [58].

K Contrasting Methodologies for VDBs: qedb vs General-Purpose SNARKs

In the introduction we motivated a modular approach based on authenticated structures by pointing out two drawbacks in general-purpose proof systems (and in circuit- or constraint-based representations): we stated that VDBs are more prone to bugs and offer a poorer developer experience. We now expand on these points and on why qedb proposes improvements in that respect. We argue that qedb mitigates these problems in at least two ways: offering a simpler *representation* and simpler *protocols*²⁷.

As a premise, recall that both qedb and several families of SNARKs—those based on (P)IOPs²⁸—can be seen, from 5000 feet, as consisting of a pipeline of this form:

representation $\xrightarrow{\text{fed into}}$ (idealized) protocol $\xrightarrow{\text{compiled into}}$ argument scheme

Both in general-purpose SNARKs and in this paper, the final compilation process is very simple; we thus focus our comparison on the two ends of the first arrow.

A “representation” of a query q in qedb directly *reflects* the SQL operations in q : it consists of a series of steps like the ones in Fig. 8, each a set/vector operation. In contrast, a circuit needs to *emulate* the relational logic through gates or constraints. The result is usually a more complicated object. For example, consider a core query operation such as set intersection. When expressed as a circuit, its most straightforward implementation is not only inefficient—it involves a quadratic number of steps—but also unnecessarily complicated. More efficient approaches add even more complexity; they may involve sorting networks and/or handling duplicates (see vSQL, Sec. VI.C [95]). An auditor or automatic tool will plausibly have an

²⁷This section will repeat and extend some of the points we made for our comparison with GKR-based schemes in Section 7.3.

²⁸A form of idealized protocol based on polynomial oracles. See, e.g., [8].

easier time spotting bugs in code for set intersection represented in the style of Fig. 8 (one constraint in our idealized VDB model) than, say, in a circuit implementing (at the very least) a sorting network. The same is arguably true for the developers taking up the task of writing the related code.

The final protocols in qedb are also simpler compared to those in common SNARKs. An idealized protocol for us consists of operations that almost directly map to the operations in a query (see Appendix E). Given a query, the steps of the protocol boil down to a handful of simple manipulations of vectors and sets. No algebraic fact is ever invoked in the protocol, which is, at its core, non-interactive. In contrast a (P)IOP typically involves running several rounds of interaction, which are usually “compressed” into one through the Fiat-Shamir heuristic (which requires particular care when implemented; see examples of documented attacks in [37]).

The checks in these idealized protocols typically involve equations over polynomials of various complexity. Protocols of this type arguably require more expertise to write, read, understand, analyze or audit (for an intuition of what we mean here, see for example the *relatively simple* idealized protocol in Figure 5 of [30]). It is our view that the (substantially structured) setting of SQL computations can leverage simpler and more easily auditable schemes without major sacrifices in efficiency.

L Other Adopted Optimizations

Batching pairing checks. In the context of verifying subvector openings, whenever we are considering the same positions for columns (e.g., in Q_{Match} or in common cases for SELECT queries) we can amortize the cost of several pairing equations for the verifier by checking only one. We do this through standard techniques. Specifically, the verifier:

- holds as input: set of indices I , commitments $\text{cm}_1, \dots, \text{cm}_\ell$, respective claimed subvector openings $\vec{v}_1, \dots, \vec{v}_\ell$, respective opening proofs $[H_1]_1, \dots, [H_\ell]_1$;
- Samples $\rho \leftarrow \mathbb{F}$;
- Computes $z^* \leftarrow \sum_j \rho^{j-1} (\text{cm} - [v_j(\tau)]_1)$, where $v_j(x)$ is the interpolation of $\vec{v}_j$ in the domain I as described in Fig. 4 (see VfSub);
- Computes $h^* \leftarrow \sum_j \rho^{j-1} [H_j]_1$;
- Checks the equation:

$$e(z^*, [1]_2) = e(h^*, [t_I]_2).$$

Delegating computation of vanishing polynomial in $\mathbb{G}_2$. In the verification algorithm for subvector opening in Fig. 4 the verifier should compute an element in $\mathbb{G}_2$ related to the vanishing polynomial $t_I(x)$. Since this operation can be expensive for a large subset I , we use a common technique that delegates this computation to the prover as follows:

- the prover sends an alleged $[t_I(\tau)]_2$ to the verifier (notice that this is a KZG commitment in $\mathbb{G}_2$ to the polynomial t_I);
- the verifier computes $t_I(x)$ on its own, samples a random challenge r and computes $y = t_I(r)$;
- the prover receives r and responds with a KZG proof π that y is the correct evaluation in r of the polynomial committed in $[t_I(\tau)]_2$;

- the verifier checks the KZG proof through a variant of the usual verification (the only change is that it will compute the pairing on the left-hand side as $e([1]_1, [t_I(\tau)]_2 - [y]_2)$.

Removing additional checks and proofs. In specific queries, combining our derived operations together from Appendix C may perform redundant checks and we optimize these out in our implementation (for example, if a first sub-protocol checks something a second sub-protocol is prescribed to check as well).